

claude-windows / f81fb2b5-e34b-427e-a8a1-a37eb4539a49

Metadata

```
- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\f81fb2b5-e34b-427e-a8a1-a37eb4539a49\subagents\workflows\wf_085a5118-2cb\agent-a99e73cb00f89e950.jsonl`
- SHA-256: `f43f7fce687e3c439e136eee5b51f6057b296b6e593795741707a3b7e1328942`
- Source modified: `2026-06-20T17:06:19+00:00`
- Imported at: `2026-07-05T16:48:22+00:00`
- project: `wf_085a5118-2cb`
- session_id: `f81fb2b5-e34b-427e-a8a1-a37eb4539a49`
```

Transcript

1. user (2026-06-20T16:58:14.070Z)

You are designing a SINGLE-FOLDER-PER-VIDEO workspace convention for a local+cloud rally pipeline (FastAPI + SQLite + ffmpeg + GPU WASB; storage layer already abstracts local/s3/gcs/gdrive via backend/storage/).

OWNER REQUIREMENT (verbatim): "for each video a single directory/folder is created and everything about it is captured only in it except its metadata in the databases, so it's clear where all the video/png/runlogs etc. for the video reside and it would be easy to repro. Replicated across local and cloud deployments."

Here is the COMPLETE map of where per-video artifacts are written today (JSON):

```
[
  {
    "area": "badminton-highlight-indexer backend pipeline segmenters",
    "write_sites": [
      {
        "artifact": "Trajectory CSV (per-detector format: Frame,Visibility,X,Y)",
        "current_location": "output/<family>/<stem>__<video_id[:12]>_predict.csv (TrackNet) or output/<family>/<stem>__<video_id[:12]>_wasb.csv (WASB)",
        "code_ref": "backend/pipeline/detectors/tracknet_runner.py:158,170 | backend/pipeline/detectors/wasb_runner.py:134,168",
        "hardcoded": false,
        "keyed_by": "video_id (MD5 namespaced into filename via video_id[:12])",
        "notes": "Written by external detectors (TrackNet/WASB in WSL) via subprocess calls; output dir passed to runner.run_predict(video_path, out_dir, video_id). Trajectory hybrid at :238 calls runner.run_predict(video_path, os.path.join('output', self.family), video_id=video_id). CSV stays on disk for parse_trajectory_csv() at :243."
      },
      {
        "artifact": "Frame cache (frame-extracted PNGs for WASB streaming fallback)",
        "current_location": "~/clips/<stem>__<video_id[:12]>_frames/ (WSL) or <scratch_dir>/<stem>__<video_id[:12]>_frames/ (native)",
        "code_ref": "backend/pipeline/detectors/wasb_runner.py:142,160,190-194 | backend/pipeline/detectors/native_wasb_runner.py:179-181,208-211",
        "hardcoded": false,
        "keyed_by": "video_id (namespaced into directory name)",
        "notes": "Created only when stream_video=False (default WSL). Cleaned up post-success unless keep_frames=True. Native WASB writes to scratch_dir or output_dir; WSL writes to ~/clips."
      },
      {
        "artifact": "Staged video file (for WSL performance, avoids /mnt/c slow path)",
        "current_location": "~/clips/<stem>__<video_id[:12]>.<ext> (WSL only)",
        "code_ref": "backend/pipeline/detectors/wasb_runner.py:94,139-141,189-194 | backend/pipeline/detectors/tracknet_runner.py:162",

```

```

        "hardcoded": false,
        "keyed_by": "video_id",
        "notes": "WSL-specific; stage_in_wsl=True (default). Deleted post-success unless
keep_frames=True."
    },
    {
        "artifact": "Candidate segments (motion-window records persisted to DB)",
        "current_location": "SQLite candidate_segments table (in-memory/disk database)",
        "code_ref": "backend/pipeline/segmenters/trajectory_hybrid.py:280-285 |
backend/pipeline/segmenters/yolo_hybrid.py:250-255",
        "hardcoded": false,
        "keyed_by": "video_id",
        "notes": "Persisted via db.add_candidate_segment(video_id, ..., stats=window_stats[i],
detection_params=detection_params). Decoupled from file system; records the raw candidate
windows + motion metadata for fine-tuning."
    },
    {
        "artifact": "Handoff clip chunks (padded candidate windows for AI validation)",
        "current_location":
"output/chunks_<provider_name>_handoff/handoff_<video_id>_<window_idx>.mp4",
        "code_ref": "backend/pipeline/segmenters/trajectory_hybrid.py:313,322 |
backend/pipeline/segmenters/yolo_hybrid.py:268,278",
        "hardcoded": true,
        "keyed_by": "video_id + window index",
        "notes": "Created at line 313 (trajectory_hybrid): handoff_dir = os.path.join('output',
f'chunks_{provider_name}_handoff'). Temporary; extracted with extract_video_chunk(), sent to AI
provider.analyze_video(), then deleted at line 338 (trajectory_hybrid) / 300 (yolo_hybrid).
Provider name (gemini, openrouter, etc.) parameterizes the directory."
    },
    {
        "artifact": "YOLO chunk extraction (per-chunk video files for person detection)",
        "current_location": "output/chunks_yolo/chunk_<video_id>_<chunk_idx>.mp4",
        "code_ref": "backend/pipeline/segmenters/yolo_hybrid.py:407,132,170",
        "hardcoded": true,
        "keyed_by": "video_id + chunk_index",
        "notes": "Created at line 407: chunks_dir = os.path.join('output', 'chunks_yolo').
Extracted per chunk for person detection. Cleaned immediately after processing (line 160, 185);
directory rmdir-ed at line 423. Pure intermediate."
    },
    {
        "artifact": "Gemini chunk extraction (per-chunk video files for full-video analysis)",
        "current_location": "output/chunks/<video_id>/chunk_<chunk_idx>_<timestamp>.mp4",
        "code_ref": "backend/pipeline/segmenters/gemini.py:23,536",
        "hardcoded": true,
        "keyed_by": "video_id (per-video subdir isolation)",
        "notes": "Via _chunk_workdir(video_id) = os.path.join('output', 'chunks', video_id) at
line 23. Chunks extracted for the full-video gemini analysis path. Per-video subdirs prevent
concurrency collisions (two video IDs don't race for the same chunk dir)."
    },
    {
        "artifact": "Temp trimmed video (debug feature for fast gemini iteration)",
        "current_location": "output/temp_trimmed_<video_id>.mp4",
        "code_ref": "backend/pipeline/segmenters/gemini.py:119-120,148",
        "hardcoded": true,
        "keyed_by": "video_id",
        "notes": "Only created if debug_duration is set. Removed at line 148 in
cleanup_temp_files(). Not for production; debug-only feature to trim long videos for testing."
    },
    {
        "artifact": "Synthetic/stub trajectory CSV (CPU-only testing path)",
        "current_location": "output/<family>/<stem>__<video_id[:12]>_stub.csv",
        "code_ref": "backend/pipeline/detectors/stub_runner.py:100-103,111-117",
        "hardcoded": false,
        "keyed_by": "video_id",
        "notes": "Created via StubDetectorRunner.run_predict() when detector_impl='stub' (or

```

RALLY_DETECTOR_IMPL=stub env var). Deterministic synthetic trajectory for regression testing without GPU/WSL. Can also replay an existing CSV if replay_csv config is set."

```
    },
    {
      "artifact": "Training synthetic trajectory CSV (from labeled rally windows)",
      "current_location": "output-relative path specified by caller (typically in training
dirs)",
      "code_ref": "backend/pipeline/detectors/stub_runner.py:124-157",
      "hardcoded": false,
      "keyed_by": "none (generated from labels_csv, output path is caller-specified)",
      "notes": "Via synth_trajectory_from_labels(labels_csv, out_csv, ...). Writes synthetic
trajectory encoding rally labels so training can run CPU-only. Not per-video; used for model
training on committed label sets."
    },
    {
      "artifact": "wasb_infer.py script (staged into WSL WASB repo for inference)",
      "current_location": "~/models/WASB-SBDT/src/wasb_infer.py (WSL) or
<repo_dir>/src/wasb_infer.py (native)",
      "code_ref": "backend/pipeline/detectors/wasb_runner.py:106-111 |
backend/pipeline/detectors/native_wasb_runner.py:109-118",
      "hardcoded": false,
      "keyed_by": "none (stateless script copy)",
      "notes": "Copied from backend/pipeline/detectors/wasb_infer.py into the WASB repo src/
so the inference subprocess can import WASB modules. Not per-video; infrastructure staging."
    },
    {
      "artifact": "Play segments (rally records persisted to DB)",
      "current_location": "SQLite play_segments table + candidate_segment links",
      "code_ref": "backend/pipeline/segmenters/trajectory_hybrid.py:366,370 |
backend/pipeline/segmenters/yolo_hybrid.py:345,352",
      "hardcoded": false,
      "keyed_by": "video_id",
      "notes": "Persisted via db.add_play_segment(video_id, start, end, ...). Confirmed AI-
validated segments (Phase 4). Linked back to source candidate via
db.link_candidate_to_segment(candidate_id, segment_id)."
    }
  ],
  "notes": "Per-video artifact mapping of the local pipeline/segmenter subsystem. All write
sites have been mapped across trajectory_hybrid.py (lines 237, 313), yolo_hybrid.py (lines 407,
268), gemini.py (lines 23, 119), and the detector runners (tracknet_runner, wasb_runner,
native_wasb_runner, stub_runner). Database writes (candidate_segments, play_segments) are
decoupled from the filesystem but recorded here for completeness. The two explicit hardcoded
paths are: (1) output/chunks_<provider_name>_handoff for AI validation clips, (2)
output/chunks_yolo for person-detection chunks. All other paths are parameterized by
family/provider_name/video_id."
},
{
  "area": "DETECTOR RUNNERS: Per-Video Artifact Write Sites",
  "write_sites": [
    {
      "artifact": "Trajectory CSV ({stem}__{vid12}_wasb.csv)",
      "current_location": "output_win_dir/{key}_wasb.csv",
      "code_ref": "backend/pipeline/detectors/wasb_runner.py:134,168,195",
      "keyed_by": "video_id",
      "hardcoded": false,
      "notes": "WSL WASB runner. Key = stem__video_id[:12]. Output dir is passed by caller.
CSV written via wsl bash command redirection (line 168: --out wsl_tilde_quote(wsl_out_dir + '/'
+ key + '_wasb.csv'))"
    },
    {
      "artifact": "Staged video copy (WSL)",
      "current_location": "wsl_stage_dir/{key}{ext}",
      "code_ref": "backend/pipeline/detectors/wasb_runner.py:94,139,142",
      "keyed_by": "video_id",
      "hardcoded": false,
```

```

    "notes": "WSL WASB runner. Default wsl_stage_dir=~/.clips (config:
indexing.wasb.wsl_stage_dir). Copied via 'cp' into WSL fs. Cleaned up on success unless
keep_frames=true (lines 189-194).",
  },
  {
    "artifact": "Frame cache directory ({key}_frames PNGs)",
    "current_location": "wsl_stage_dir/{key}_frames/",
    "code_ref": "backend/pipeline/detectors/wasb_runner.py:142,160,190",
    "keyed_by": "video_id",
    "hardcoded": false,
    "notes": "WSL WASB runner PNG extraction. Default wsl_stage_dir=~/.clips. Created by
--frames_out_dir arg (line 160). Skipped if stream_video=true (line 157). Cleaned via _wsl_rm_rf
on success unless keep_frames=true (lines 189-194).",
  },
  {
    "artifact": "Detector cache ({stem}__{vid12}_wasb_wasbcache/det_raw.jsonl)",
    "current_location": "{output_win_dir}/{key}_wasb_wasbcache/det_raw.jsonl",
    "code_ref": "backend/pipeline/detectors/wasb_infer.py:90-94,438",
    "keyed_by": "video_id",
    "hardcoded": false,
    "notes": "Default cache dir is next to trajectory CSV (from default_cache_dir at line
90-94). Path = dirname(out_csv) + stem(out_csv) + '_wasbcache'. Incremental detector pass cache,
resumable across reboots.",
  },
  {
    "artifact": "Detector cache manifest ({stem}__{vid12}_wasb_wasbcache/manifest.json)",
    "current_location": "{output_win_dir}/{key}_wasb_wasbcache/manifest.json",
    "code_ref": "backend/pipeline/detectors/wasb_infer.py:90-94,122,465",
    "keyed_by": "video_id",
    "hardcoded": false,
    "notes": "Tracks frames_dir, weights, sport, frames_in, frames_total, device for cache
reuse check. Atomic write (tmp+rename) at line 122.",
  },
  {
    "artifact": "Trajectory CSV ({stem}__{vid12}_predict.csv or {stem}_predict.csv)",
    "current_location": "output_win_dir/{key}_predict.csv",
    "code_ref": "backend/pipeline/detectors/tracknet_runner.py:157-170,202,210",
    "keyed_by": "video_id",
    "hardcoded": false,
    "notes": "TrackNet WSL runner. Key = stem__video_id[:12] (line 157). If no staging, key
= stem (line 170). CSV path is from WSL predict.py output (derived from staged video stem).",
  },
  {
    "artifact": "Staged video copy (WSL)",
    "current_location": "wsl_stage_dir/{key}{ext}",
    "code_ref": "backend/pipeline/detectors/tracknet_runner.py:162,222-232",
    "keyed_by": "video_id",
    "hardcoded": false,
    "notes": "TrackNet WSL runner. Default wsl_stage_dir=~/.clips (config:
indexing.tracknet.wsl_stage_dir). Copied via _stage_video (lines 222-232). Cleaned on success
unless keep_frames=true (lines 206-209).",
  },
  {
    "artifact": "Trajectory CSV ({stem}__{vid12}_predict.csv) [native]",
    "current_location": "output_dir/{key}_predict.csv",
    "code_ref": "backend/pipeline/detectors/native_wasb_runner.py:165,172,185",
    "keyed_by": "video_id",
    "hardcoded": false,
    "notes": "Native WASB runner (no WSL). Key = stem__video_id[:12] (line 164). Output dir
is passed by caller, output_dir = abspath(output_win_dir) at line 158.",
  },
  {
    "artifact": "Frame cache directory ({key}_frames PNGs) [native]",
    "current_location": "scratch_dir or output_dir/{key}_frames/",
    "code_ref": "backend/pipeline/detectors/native_wasb_runner.py:179-181,208-211",
  },

```

```

        "keyed_by": "video_id",
        "hardcoded": false,
        "notes": "Native WASB runner. Scratch resolution: env WASB_SCRATCH or RALLY_SCRATCH or
TMPDIR or output_dir (line 179). Skipped if stream_video=true. Cleaned via _rm_rf on success
unless keep_frames=true (lines 208-211).",
    },
    {
        "artifact": "Detector cache ({stem}__{vid12}_wasb_wasbcache/) [native]",
        "current_location": "output_dir/{key}_wasb_wasbcache/",
        "code_ref": "backend/pipeline/detectors/wasb_infer.py:90-94",
        "keyed_by": "video_id",
        "hardcoded": false,
        "notes": "Native WASB runner uses same wasb_infer.py. Default cache next to output CSV.
Path = dirname(out_csv) + stem(out_csv) + '_wasbcache'."
    },
    {
        "artifact": "Trajectory CSV ({stem}__{vid12}_stub.csv) [replay/synth]",
        "current_location": "output_win_dir/{key}_stub.csv",
        "code_ref": "backend/pipeline/detectors/stub_runner.py:103,107,116",
        "keyed_by": "video_id",
        "hardcoded": false,
        "notes": "CPU stub runner (no GPU/WSL). Key = stem__video_id[:12] (line 102). Can either
replay existing CSV (line 106) or synthesize deterministic trajectory (lines 111-116).",
    }
],
"config_or_env_knobs": [
    "indexing.wasb.wsl_stage_dir",
    "indexing.wasb.keep_frames",
    "indexing.wasb.stream_video",
    "indexing.wasb.min_free_gb",
    "indexing.tracknet.wsl_stage_dir",
    "indexing.tracknet.keep_frames",
    "indexing.tracknet.stage_in_wsl",
    "indexing.tracknet.min_free_gb",
    "WASB_SCRATCH",
    "RALLY_SCRATCH",
    "TMPDIR",
    "WASB_WEIGHTS",
    "WASB_REPO_DIR",
    "WASB_PYTHON",
    "WASB_DEVICE"
],
"notes": "SCRATCH DIR RESOLUTION (native_wasb_runner.py:87): env(\"WASB_SCRATCH\") or
env(\"RALLY_SCRATCH\") or env(\"TMPDIR\") or \"\". WASB_SCRATCH takes precedence, falls back to
RALLY_SCRATCH, then TMPDIR, then empty string (output_dir used instead). CACHE HYGIENE: (1) On
run_predict success, staged video + frame cache are deleted unless keep_frames=true (WSL:
_wsl_rm_rf at wasb_runner.py:194; native: _rm_rf at native_wasb_runner.py:211). (2) On any
earlier failure, cleanup is skipped to preserve cache for resumable re-run. (3) Streaming mode
(stream_video=true) never writes PNGs, so only the staged video is deleted. CSV NAMING:
{stem}__{video_id[:12]}_RUNNER.csv where RUNNER is 'wasb' (wasb_runner/native_wasb_runner) or
'predict' (tracknet_runner) or 'stub' (stub_runner). CACHE KEYS (wasb_infer.py:137-154):
manifest_compatible checks frames_dir (abspath), weights path, sport, frames_in, frames_total,
and device. Device defaults to 'cuda' for backward compat (line 153). WSL paths: to_wsl_mnt_path
translates Windows C:\\\\x to /mnt/c/x; wsl_tilde_quote preserves ~ for expansion while quoting
the rest.\""
},
{
    "area": "badminton-highlight-indexer WORKER + STORAGE + BUCKET artifact write mapping",
    "write_sites": [
        {
            "artifact": "intervals.csv (rallies per-video)",
            "current_location": "scratch/outputs/{video_id}/intervals.csv",
            "code_ref": "backend/worker/__main__.py:146",
            "hardcoded": false,
            "keyed_by": "video_id",
        }
    ]
}

```

```

    "notes": "CSV written via _write_intervals_csv(); then pushed to bucket via
storage.put(). Schema: start, end, shot_count, ending_reason (decimal seconds)"
},
{
    "artifact": "report.json (per-video indexing report)",
    "current_location": "scratch/outputs/{video_id}/report.json",
    "code_ref": "backend/worker/__main__.py:147",
    "hardcoded": false,
    "keyed_by": "video_id",
    "notes": "JSON written via _write_report_json(); contains: video_id, status,
segment_count, segments[][start, end]. Then pushed to bucket via storage.put()"
},
{
    "artifact": "outputs/{video_id}/* (cmd_infer output prefix)",
    "current_location": "{out_uri}/outputs/{video_id}/{filename}",
    "code_ref": "backend/worker/__main__.py:149-158",
    "hardcoded": false,
    "keyed_by": "video_id",
    "notes": "storage.put() loop iterates local artifacts in scratch/outputs/{video_id}/ and
pushes each to {out_uri}/outputs/{video_id}/{fn}. --out-uri arg is the prefix base"
},
{
    "artifact": "sports_indexer.db (SQLite3 per-run database)",
    "current_location": "scratch/{config.output.db_name}",
    "code_ref": "backend/worker/__main__.py:107",
    "hardcoded": false,
    "keyed_by": "none (single per run)",
    "notes": "Database(db_path) constructor creates/opens SQLite3 at given path. Default:
config.output.db_name='sports_indexer.db'. Stores videos, play_segments, events tables. Uses WAL
mode. NOT pushed to bucket by worker (local artifact only)"
},
{
    "artifact": "in.mp4 (downloaded video temp)",
    "current_location": "scratch/in.mp4",
    "code_ref": "backend/worker/__main__.py:98",
    "hardcoded": true,
    "keyed_by": "video_id",
    "notes": "storage.fetch(args.video_uri, scratch/in.mp4, ...) pulls video to local disk
for processing. Identity passthrough for local://, downloads for s3://, gcs://"
},
{
    "artifact": "labels/ (training input ? golden CSVs)",
    "current_location": "scratch/labels/{filename}",
    "code_ref": "backend/worker/__main__.py:282",
    "hardcoded": false,
    "keyed_by": "none (all pulled from corpus)",
    "notes": "cmd_train: storage.fetch(uri, os.path.join(labels_dir, fname), ...) for each
label_uri from storage.list_uris('{corpus}/labels/', ...). Local scratch copy for processing"
},
{
    "artifact": "trajectories/ (detector or synth)",
    "current_location": "scratch/trajectories/{name}_traj.csv",
    "code_ref": "backend/worker/__main__.py:298-309",
    "hardcoded": false,
    "keyed_by": "name (video stem)",
    "notes": "cmd_train: either real detector (runner.run_predict) or synth stub
(synth_trajectory_from_labels). Stored in local traj_dir. CSV with frame trajectory data"
},
{
    "artifact": "videos/ (training input ? source videos for detector)",
    "current_location": "scratch/videos/{filename}",
    "code_ref": "backend/worker/__main__.py:288",
    "hardcoded": false,
    "keyed_by": "none (pulled from corpus)",
    "notes": "cmd_train --trajectory-source detector only: storage.fetch(vuri, ...) for each

```

```

video in storage.list_uris('{corpus}/videos/', ...). Downloaded for detector inference"
},
{
  "artifact": "manifest.json (training corpus spec)",
  "current_location": "scratch/manifest.json",
  "code_ref": "backend/worker/__main__.py:314-316",
  "hardcoded": true,
  "keyed_by": "none (single per training run)",
  "notes": "JSON list of {name, trajectory, labels, fps, frame_width} specs. Written
locally before handoff to training.gen0.harness subprocess"
},
{
  "artifact": "weights/{version}/* (artifact bundle ? training output)",
  "current_location": "{out_uri}/weights/{version}/{filename}",
  "code_ref": "backend/worker/__main__.py:338-340",
  "hardcoded": false,
  "keyed_by": "version",
  "notes": "cmd_train: training.gen0.harness writes to out_dir/{version}/. Worker then
storage.put() each file to {out_uri}/weights/{version}/{fn}. Contains weights.json +
manifest.json"
},
{
  "artifact": "weights.json (learned model)",
  "current_location": "out_dir/{version}/weights.json",
  "code_ref": "training/gen0/harness.py:157-162",
  "hardcoded": false,
  "keyed_by": "version",
  "notes": "Logistic regression classifier + calibration (threshold, smooth, merge_gap,
min_dur) + feature_schema_version. JSON payload written locally, then pushed by worker"
},
{
  "artifact": "manifest.json (artifact metadata)",
  "current_location": "out_dir/{version}/manifest.json",
  "code_ref": "training/gen0/harness.py:197-199",
  "hardcoded": false,
  "keyed_by": "version",
  "notes": "Artifact identity: name, version, git_sha, created_at, weights_sha256.
Training lineage: corpus manifest, videos, per-video labels hash. Eval results (LOVO F1, per-
video scores). Attestation block. Ship-gate verdict"
},
{
  "artifact": "floor.json (baseline for gate comparison)",
  "current_location": "scratch/floor.json (temp)",
  "code_ref": "backend/worker/__main__.py:324-326",
  "hardcoded": false,
  "keyed_by": "none (input baseline)",
  "notes": "cmd_train --floor: if ':' in path, storage.fetch() to scratch; else treated
as local path. Used by training harness for gate_verdict() comparison"
},
{
  "artifact": "report (obsreport soft dependency)",
  "current_location": "(config.report.output_dir or None)",
  "code_ref": "backend/reporting/__init__.py:72-73",
  "hardcoded": false,
  "keyed_by": "video_id",
  "notes": "emit_indexer_report() calls harvest.record_run().write(output_dir=...). SOFT
dependency (no-op if obsreport not installed). May write per-video telemetry if enabled"
}
},
"config_or_env_knobs": [
  "--scratch DIR (worker)",
  "--out-uri PREFIX (worker infer/train)",
  "--corpus-uri PREFIX (worker train)",
  "--config config.json (load_config)",
  "config.output.output_dir (OutputConfig default: 'output')",

```

```

    "config.storage.backend (StorageConfig default: 'local')",
    "config.storage.output_root (fallback to output.output_dir if empty)",
    "config.storage.s3/gcs/gdrive/*.* (per-backend settings: endpoint_url, region, mount_root,
etc.)",
    "config.report.output_dir (reporting optional)",
    "GDRIVE_MOUNT_ROOT env (gdrive backend mount auto-detect)",
    "AWS_ENDPOINT_URL_S3 env (S3 storage endpoint override)",
    "GOOGLE_APPLICATION_CREDENTIALS env (GCS auth)",
    "AWS_ACCESS_KEY_ID, AWS_SECRET_ACCESS_KEY, AWS_SESSION_TOKEN env (S3 creds)"
],
"notes": "WORKER ARCHITECTURE: cmd_infer and cmd_train are orchestrators that compose
storage layer (fetch/put/list_uris) + segmenters (process_video) + reporting
(emit_indexer_report) + training harness subprocess. Both operations use a local scratch
directory (--scratch or tempdir) as the working directory; config.output.output_dir is
overridden to scratch inside cmd_infer to keep output_dir semantics local-scoped. STORAGE LAYER
(backend/storage/): URI-driven, lazy-loading cloud SDKs. fetch() is identity passthrough for
local/bare paths, downloads for s3/gcs, mount-read for gdrive. put() copies local file to remote
URI. list_uris() yields all objects under prefix (used by cmd_train to discover corpus
labels/videos). BUCKET LAYOUT: Prefix-based on storage backend. Infer writes
outputs/{video_id}/{intervals.csv,report.json}. Train writes
weights/{version}/{weights.json,manifest.json}. Per-video keying is via MD5 (compute_video_id)
in infer; training is version-keyed (--version arg). LOCAL BACKEND: No secrets, bare filesystem
paths or local:// URIs. S3/GCS/GDrive: Credentials from environment chains (boto3 default,
Application Default Credentials, Google Drive for Desktop mount). Config carries only
endpoints/regions/paths. DATABASE: Not pushed to bucket ? SQLite3 sports_indexer.db stays in
scratch/output_dir only. WAL mode; auto-detects if FS doesn't support (networked paths).
DEPENDENCIES: obsreport (soft) for per-video telemetry; training.gen0.harness (imported only in
cmd_train subprocess call, not at module load). IDEMPOTENCE: video_id (MD5) is the join key;
infer is idempotent on outputs/{video_id}/. Training is version-keyed; weights/{version}/ is the
identity."
},
{
    "area": "Per-video artifact write sites and database schema mapping for badminton-highlight-
indexer",
    "write_sites": [
        {
            "artifact": "Database (SQLite) ? core relational storage",
            "current_location": "output/sports_indexer.db (default; configurable via output.db_name
in config.json)",
            "code_ref": "backend/infrastructure/database.py:10, backend/config/models.py:230-233",
            "hardcoded": false,
            "keyed_by": "video_id (MD5 hash)",
            "notes": "SQLite database. Schema version 5. Contains videos, play_segments,
candidate_segments, events, jobs, user_models tables."
        },
        {
            "artifact": "Per-video metadata ? videos table",
            "current_location": "sports_indexer.db:videos table",
            "code_ref": "backend/infrastructure/database.py:112-120, backend/main.py:170",
            "hardcoded": false,
            "keyed_by": "video_id (MD5 hash of filepath)",
            "notes": "PURE METADATA. Fields: id (PK), filepath (NOT NULL), processed_at (TIMESTAMP),
status (NOT NULL: 'processed'/'failed'), validation_results (JSON), user_id (NULL=local). No
file paths reference."
        },
        {
            "artifact": "Play segments ? play_segments table",
            "current_location": "sports_indexer.db:play_segments table",
            "code_ref": "backend/infrastructure/database.py:123-135, backend/main.py:194",
            "hardcoded": false,
            "keyed_by": "video_id (FK to videos.id)",
            "notes": "PURE METADATA. Per-rally detected segments. Fields: id, video_id, start_time,
end_time, duration, server, receiver, metadata (JSON: detector, shot_count, etc.). No file
paths."
        }
    ],

```



```

{
  "artifact": "Candidate segments ? candidate_segments table (v1)",
  "current_location": "sports_indexer.db:candidate_segments table",
  "code_ref": "backend/infrastructure/database.py:154-173, backend/main.py:194",
  "hardcoded": false,
  "keyed_by": "video_id (FK, detector name)",
  "notes": "PURE METADATA. Pre-confirmation raw detections per detector. Fields: id,
video_id, detector, chunk_index, start_time, end_time, duration, confidence, stats (JSON:
detector-specific metrics), detection_params (JSON), confirmed_segment_id, human_label, notes,
created_at, user_id (NULL=local).",
},
{
  "artifact": "Events ? events table",
  "current_location": "sports_indexer.db:events table",
  "code_ref": "backend/infrastructure/database.py:138-148",
  "hardcoded": false,
  "keyed_by": "segment_id (FK to play_segments.id)",
  "notes": "PURE METADATA. Rally event timestamps (e.g. smash, foul). Fields: id,
segment_id, timestamp, type, player, details (JSON). No file paths."
},
{
  "artifact": "Async jobs ? jobs table (v2)",
  "current_location": "sports_indexer.db:jobs table",
  "code_ref": "backend/infrastructure/database.py:187-205, backend/main.py:263-267,
backend/api/job_worker.py:72-97",
  "hardcoded": false,
  "keyed_by": "job_id (UUID hex)",
  "notes": "Job execution state (queued|running|done|failed). Fields: id (PK), video_path,
video_id, segmenter, player_pool (JSON), max_frames, status, progress, error, result (JSON: full
response dict including report paths), created_at, started_at, finished_at, policy (JSON:
ExecutionPolicy), next_poll_at (TIMESTAMP).",
},
{
  "artifact": "Per-user models ? user_models table (v5)",
  "current_location": "sports_indexer.db:user_models table",
  "code_ref": "backend/infrastructure/database.py:256-285",
  "hardcoded": false,
  "keyed_by": "user_id (NULL=local), version",
  "notes": "PURE METADATA. Per-user ML models (inline classifier_json, not file-backed).
Fields: id, user_id, version, baseline_version, feature_schema_hash, fusion_config (JSON),
classifier_json (JSON: LogisticRegression.to_dict()), promotion_evidence (JSON),
n_videos_at_fit, is_active, created_at."
},
{
  "artifact": "Evaluation CSV outputs ? rally segmentation scores",
  "current_location": "output/<name>_<tuned|baseline>_segments.csv (hardcoded default:
output/wasb_tuned_segments.csv)",
  "code_ref": "backend/eval/export_wasb_segments.py:70-85, 139-179",
  "hardcoded": true,
  "keyed_by": "none (file pattern-based)",
  "notes": "CSV export from calibrate_wasb / export_wasb_segments. Schema: rally_number,
start_time, end_time, duration_sec, mm:ss. Per-video trajectory ? windowing ? CSV."
},
{
  "artifact": "Evaluation comparison CSV ? tuned vs golden",
  "current_location": "output/<name>_vs_golden.csv (hardcoded default:
output/wasb_tuned_segments_vs_golden.csv)",
  "code_ref": "backend/eval/export_wasb_segments.py:125-135",
  "hardcoded": true,
  "keyed_by": "none (file pattern-based)",
  "notes": "CSV: matched (TP), misses (FN), extra (FP), precision, recall, F1 vs golden
ground truth."
},
{
  "artifact": "Run regression baseline ? JSON snapshot",

```

```

        "current_location": "output/regression_baselines.json (default:
backend/eval/regression.py:DEFAULT_BASELINE_PATH)",
        "code_ref": "backend/eval/regression.py:44-48, 96-116, run_regression.py:43, 71-75",
        "hardcoded": true,
        "keyed_by": "video_id, stage, detector, iou_threshold",
        "notes": "PURE METADATA. Regression test snapshots: per-video {video_id, stage,
precision, recall, f1, iou_threshold, detector, gt_fingerprint, timestamp}. Used as gate
baseline."
    },
    {
        "artifact": "Golden rally features ? per-video JSON",
        "current_location": "output/<video_name>_golden_features.json (pattern-based)",
        "code_ref": "backend/eval/rally_seg_eval.py:15-51, 32-33",
        "hardcoded": false,
        "keyed_by": "video name",
        "notes": "PURE METADATA. Golden test set. Fields: name, fps, frame_width,
trajectory_path (FILE PATH reference to .csv), gts (list of [start, end] intervals)."
    },
    {
        "artifact": "Golden manifest ? all test videos metadata",
        "current_location": "output/human_lovo_manifest.json (hardcoded default)",
        "code_ref": "backend/eval/rally_seg_eval.py:32, 47-60",
        "hardcoded": true,
        "keyed_by": "video name (list of dicts)",
        "notes": "PURE METADATA. List of golden test videos: {name, trajectory_path, fps,
frame_width, [court_type]}. Trajectory files are .csv PATHS."
    },
    {
        "artifact": "Highlight reel (compiled video)",
        "current_location": "output/<basename(output_filename)> (from /api/compile or scripts)",
        "code_ref": "backend/pipeline/stitcher/compiler.py:326, 303-407,
backend/main.py:342-396, config/models.py:229",
        "hardcoded": false,
        "keyed_by": "video_id (implicit, stored in jobs.result[reports])",
        "notes": "VIDEO FILE (.mp4). Output of VideoCompiler.compile_highlights(). Stitched from
per-segment clips extracted via ffmpeg. Path stored in jobs.result['reports'] and /api/compile
response."
    },
    {
        "artifact": "Per-segment extracted clips (temporary, intermediate)",
        "current_location": "tempfile.TemporaryDirectory(dir=output_dir)/clip_<idx>.mp4",
        "code_ref": "backend/pipeline/stitcher/compiler.py:233, 340, 348-351",
        "hardcoded": false,
        "keyed_by": "segment index (idx) within compile operation",
        "notes": "TRANSIENT intermediate. Created during compile_highlights(), cleaned up after
concat. Per-segment re-encode with optional watermark."
    },
    {
        "artifact": "Observability report ? per-video JSON",
        "current_location": "output/<video_id>_report.json (or configured report.output_dir)",
        "code_ref": "backend/reporting/__init__.py:72-74,
backend/reporting/harvest.py:record_run().write(), backend/main.py:230-237",
        "hardcoded": false,
        "keyed_by": "video_id (MD5)",
        "notes": "PURE METADATA. Obsreport observability report (soft dependency). Staged
signals: validation, segmentation, AI handoff, stitching, metadata. Paths returned in
jobs.result['reports']."
    },
    {
        "artifact": "Job result JSON ? full response stored in DB",
        "current_location": "sports_indexer.db:jobs.result (TEXT column, JSON-serialized)",
        "code_ref": "backend/main.py:219-226, backend/api/job_worker.py:85-90,
backend/infrastructure/database.py:520-528",
        "hardcoded": false,
        "keyed_by": "job_id (UUID hex)",

```

```

    "notes": "PURE METADATA + FILE PATH REFERENCES. Stored as JSON in jobs table. Contains:
{status, message, results (validation), play_segments (metadata), reports (dict of report
paths)}."
},
{
    "artifact": "Run logs ? processing execution logs",
    "current_location": "Streamed to stdout/stderr; can be captured to
<video_name>_extract.log (manual via shell redirection)",
    "code_ref": "backend/main.py:12-17 (logging.basicConfig), backend/utils/logging_setup.py
(if used), scripts/run_process_and_stitch.py (example capture)",
    "hardcoded": false,
    "keyed_by": "video_path (implicit in log context)",
    "notes": "NOT persisted by default (streaming logger). Manual capture: `python script.py
> output/video_extract.log 2>&1`. Backend logs via logging module at INFO level."
},
{
    "artifact": "Chunk intermediate videos (WASB/TrackNet staging)",
    "current_location": "output/chunks/<video_id>/chunk_<video_id>_<idx>.mp4 (or WSL
wsl_stage_dir per config)",
    "code_ref": "backend/pipeline/detectors/wasb_runner.py (WSL staging),
backend/pipeline/detectors/tracknet_runner.py (WSL staging), config/models.py:51-52, 68-79",
    "hardcoded": false,
    "keyed_by": "video_id, chunk_index",
    "notes": "TRANSIENT intermediates. Split video into chunks for detector input. Auto-
deleted on success (keep_frames=false); kept on failure for resume."
},
{
    "artifact": "WASB trajectory CSV ? shuttle tracking output",
    "current_location": "Produced by WASB detector in WSL or native; referenced in golden
manifest trajectory_path field",
    "code_ref": "backend/eval/calibrate_wasb.py:44-56 (parse_trajectory_csv),
backend/eval/export_wasb_segments.py:139-179 (consume trajectory)",
    "hardcoded": false,
    "keyed_by": "video_id (implicit in trajectory CSV filename)",
    "notes": "CSV: Frame, Visibility, X, Y (shuttle coordinates). Produced by WASB detector,
consumed by calibrate_wasb / export_wasb_segments to generate rally windows."
}
},
"config_or_env_knobs": [
    "output.output_dir (backend/config/models.py:233, default 'output')",
    "output.db_name (backend/config/models.py:230, default 'sports_indexer.db')",
    "stitching.begin_padding / end_padding (backend/config/models.py:254-255, default 0.5)",
    "indexing.wasb.keep_frames (backend/config/models.py:77, default false)",
    "indexing.tracknet.keep_frames (backend/config/models.py:48, default false)",
    "report.output_dir (backend/reporting/__init__.py:72, optional override)",
    "storage.output_root (backend/config/models.py:304, fallback to output.output_dir)"
],
"notes": "DATABASE SCHEMA SUMMARY: The core storage is SQLite (sports_indexer.db) with 8
tables. All rally metadata, detection stats, job state, and per-user models are STORED AS PURE
METADATA in the database. FILE REFERENCES: Only two places store file paths: (1) videos.filepath
(source video), and (2) jobs.result['reports'] (dict of written report paths). ARTIFACT WRITE
FLOWS: (1) EVAL: trajectory CSV ? calibrate_wasb / export_wasb_segments ?
output/<name>_segments.csv (+ optional _vs_golden.csv). (2) REELS/HIGHLIGHTS: play_segments
metadata + video filepath ? VideoCompiler.compile_highlights() ? output/<basename>.mp4 (+ per-
segment transient clips in tempdir). (3) RUN LOGS: Streamed via Python logging module (not auto-
persisted; manual shell redirect to output/<name>_extract.log). (4) OBSREPORT: Harvested from DB
+ config ? obsreport.RunRecorder.write() ? output/<video_id>_report.json. (5) JOB STATE:
Execution state + result dict stored in jobs table; job.result['reports'] contains paths
written. CONFIG KNOBS: output.output_dir sets base directory; report.output_dir can override
report location; storage backend can remap all outputs (S3/GCS/GDrive). HARDCODED PATTERNS: WASB
eval defaults use 'output/' prefix (export_wasb_segments.py:140, 165-168); golden fixtures load
from 'output/human_lovo_manifest.json' (rally_seg_eval.py:32).\"
}
]

```

Design the target convention. Key constraints/anchors to honor:

- video_id = MD5 of the file (backend/utils/hashing.py::compute_video_id) is the natural per-video key.
- The storage layer already gives local/bucket parity via URIs (local:// / gcs:// / s3:// / gdrive://). The workspace root should be a storage URI/prefix so LOCAL and CLOUD deployments use the SAME code path and the SAME relative layout (only the root differs).
- DB stores METADATA ONLY (+ a reference/URI to the workspace), never duplicating the artifacts.
- Everything for a video (the fetched video copy or a pointer, PNG frame cache, trajectory CSV, candidate CSV, intervals.csv, report.json, run log, reel mp4) lives under ONE root <workspace>/<video_id>/... ? easy to find, easy to repro, easy to delete (cache hygiene).
- Be honest about back-compat: many sites hardcode "output"/"output/<family>" and the worker uses ad-hoc temp scratch + outputs/<video_id>/. List EXACT code changes (file:line) to converge them on a single workspace-path helper.
- Propose a small helper (e.g. backend/utils/workspace.py building <root>/<video_id>/<artifact>) and which call sites adopt it.

Return the convention, the layout tree, what stays in the DB, the full code-change list (file:line -> change), a phased implementation plan (smallest-safe-first; default-OFF or back-compat where needed), back-compat risks, and any genuine open questions for the owner.

2. assistant (2026-06-20T16:58:18.287Z)

I'll design this convention. Let me first verify the current code against the provided map by reading the key files, since the map needs to be accurate down to file:line.

3. user (2026-06-20T16:58:21.415Z)

Your configuration specifies to merge with the ref 'refs/heads/docs/deploy-report-polish' from the remote, but no such ref was fetched.

4. user (2026-06-20T16:58:21.421Z)

```
1      """Canonical file-identity hashing for the indexer.
2
3      `video_id` is the MD5 of the *whole* video file. Historically each endpoint
4      (`backend/main.py`, `run_eval.py`, `scripts/run_phase3_eval.py`,
5      `scripts/run_process_and_stitch.py`)
6      defined its own `get_file_md5` with a different read-chunk size (64KB / 8KB / 1MB).
7      The hash values agreed (same bytes hashed), but the duplicated code was a footgun:
8      any future change to one copy would silently diverge the DB key.
9
10     This module is the single source of truth. Import `compute_video_id` everywhere a
11     video needs to be keyed.
12     """
13     from __future__ import annotations
14
15     import hashlib
16
17     # 64 KB matches the value used by the live pipeline (backend/main.py) before this
18     # was centralized. The chunk size does not affect the resulting digest ? it only
19     # bounds memory use while streaming large files.
20     _CHUNK_SIZE = 65536
21
22
23     def compute_video_id(filepath: str) -> str:
24         """Return the MD5 hex digest of the whole file ? the canonical video_id.
25
26         Streams the file in fixed-size chunks so multi-gigabyte recordings never load
27         into memory at once.
28         """
29         hasher = hashlib.md5()
30         with open(filepath, "rb") as f:
31             for chunk in iter(lambda: f.read(_CHUNK_SIZE), b""):
```

```

32         hasher.update(chunk)
33     return hasher.hexdigest()
34

```

5. assistant (2026-06-20T16:58:25.350Z)

Let me read the actual code files referenced in the map to verify the line numbers and current behavior before designing.

6. user (2026-06-20T16:58:26.318Z)

```

1     """Worker dispatcher ? pull from storage ? run the pipeline ? push outputs back.
2
3     python -m backend.worker infer --video-uri <uri> --out-uri <prefix> \
4         [--segmenter wasb_hybrid] [--config config.json] [--set k=v ...] [--scratch DIR]
5     python -m backend.worker train ...          # Gen-0 train-from-bucket (follow-up)
6
7     The URI scheme selects the storage backend via ``backend.storage`` (a bare path /
8     ``local://``
9     is an identity passthrough; ``s3://`` / ``gcs://`` is an object store), so existing
10    local runs
11    are byte-for-byte unchanged. Combine with ``RALLY_DETECTOR_IMPL=stub`` and
12    ``--set indexing.skip_ai_handoff=true`` for a **secret-free, GPU-free** run (no Gemini
13    key) ?
14    the path validated end-to-end on DigitalOcean (``docs/SETUP_NEW_MACHINE.md``).
15
16    This is a thin orchestrator: it composes the SAME building blocks the ``backend.main``
17    CLI uses
18    (validator registry ? segmenter ? report), never a forked pipeline.
19    """
20    from __future__ import annotations
21
22    import argparse
23    import csv
24    import json
25    import os
26    import sys
27    import tempfile
28    from typing import Any, List
29
30    from backend import storage
31    from backend.config import load_config
32    from backend.infrastructure.database import Database
33    from backend.pipeline.segmenters.base import segmenter_registry
34    from backend.pipeline.validators.base import registry as validator_registry
35    from backend.reporting import emit_indexer_report
36    from backend.results import Failure
37    from backend.utils.hashing import compute_video_id
38
39
40    def _coerce(v: str) -> Any:
41        """Coerce a ``--set`` string value to bool/int/float where it round-trips, else
42        str."""
43        low = v.lower()
44        if low in ("true", "false"):
45            return low == "true"
46        for cast in (int, float):
47            try:
48                return cast(v)
49            except ValueError:
50                pass
51        return v
52
53    def _apply_overrides(config: Any, sets: List[str]) -> None:

```

```

50         """Apply ``a.b.c=value`` overrides onto the typed config (e.g.
indexing.skip_ai_handoff=true)."""
51     for kv in sets or []:
52         key, sep, val = kv.partition("=")
53         if not sep:
54             raise ValueError(f"--set expects k=v, got {kv!r}")
55         parts = key.split(".")
56         obj = config
57         for p in parts[:-1]:
58             obj = getattr(obj, p)
59         setattr(obj, parts[-1], _coerce(val))
60
61
62     def _write_intervals_csv(segments: List[dict], path: str) -> None:
63         """Decimal-second ``[start,end)`` rallies + shot count / ending reason ? the join-
friendly
64         artifact (same schema as the rally-annotator labels)."""
65         with open(path, "w", newline="") as f:
66             w = csv.writer(f)
67             w.writerow(["start", "end", "shot_count", "ending_reason"])
68             for s in segments:
69                 w.writerow([
70                     f'{float(s.get("start_time", 0.0)):.3f}',
71                     f'{float(s.get("end_time", 0.0)):.3f}',
72                     s.get("shot_count", ""),
73                     s.get("ending_reason", s.get("shot_count_confidence", "")),
74                 ])
75
76
77     def _write_report_json(video_id: str, segments: List[dict], status: str, path: str) ->
None:
78         with open(path, "w") as f:
79             json.dump({
80                 "video_id": video_id,
81                 "status": status,
82                 "segment_count": len(segments),
83                 "segments": [{ "start": float(s.get("start_time", 0.0)),
84                               "end": float(s.get("end_time", 0.0)) } for s in segments],
85             }, f, indent=2)
86
87
88     def cmd_infer(args: argparse.Namespace) -> int:
89         config = load_config(args.config) if args.config else load_config()
90         _apply_overrides(config, args.set)
91         storage_cfg = config.storage.model_dump()
92
93         scratch = args.scratch or tempfile.mkdtemp(prefix="rally-worker-")
94         os.makedirs(scratch, exist_ok=True)
95         config.output.output_dir = scratch
96
97         # 1. PULL ? local:// / bare path = identity passthrough; s3:// / gcs:// = download
to scratch.
98         local_video = storage.fetch(args.video_uri, os.path.join(scratch, "in.mp4"),
config=storage_cfg)
99         print(f"[worker] pulled {args.video_uri} -> {local_video}")
100
101         # 2. IDENTITY ? whole-file MD5 (the labels?video join key; pulled bytes must be
byte-identical).
102         video_id = compute_video_id(local_video)
103
104         # 3. VALIDATE (same registry as the main CLI).
105         val_results, val_context = validator_registry.run_all_with_context(local_video,
config)
106         failures = [r for r in val_results if not r.passed]
107         db = Database(os.path.join(scratch, config.output.db_name))

```

```

108     db.add_video(video_id, local_video, "failed" if failures else "processed",
109                 [r.model_dump() for r in val_results])
110
111     segments: List[dict] = []
112     segmenter_actual = None
113     segmentation_ran = False
114     status = "failed"
115     try:
116         if failures:
117             print("[worker] validation FAILED: "
118                   + "; ".join(f"{r.validator_name}: {r.message}" for r in failures))
119             return 2
120         seg_name = args.segmenter or config.indexing.default_segmenter
121         segmenter_cls = segmenter_registry.get(seg_name)
122         if not segmenter_cls:
123             print(f"[worker] unknown segmenter: {seg_name!r} "
124                   f"(have {list(segmenter_registry.list_available())})")
125             return 2
126         segmenter_actual = seg_name
127         result = segmenter_cls(db, config).process_video(video_id, local_video,
128 max_frames=args.max_frames)
129         segmentation_ran = True
130         if isinstance(result, Failure):
131             print(f"[worker] segmenter FAILED: {result.message}")
132             return 3
133         segments = result.value if hasattr(result, "value") else result
134         status = "success"
135     finally:
136         # Best-effort per-video observability report (never raises) ? parity with the
137 main CLI.
138         emit_indexer_report(
139             db=db, video_id=video_id, video_path=local_video, config=config,
140             val_results=val_results, val_context=val_context,
141             segmenter_requested=args.segmenter, segmenter_actual=segmenter_actual,
142             was_reingest=False, segmentation_ran=segmentation_ran,
143         )
144
145     # 5. SERIALIZE + 6. PUSH (outputs/<video_id>/?). idempotent on video_id.
146     out_local = os.path.join(scratch, "outputs", video_id)
147     os.makedirs(out_local, exist_ok=True)
148     _write_intervals_csv(segments, os.path.join(out_local, "intervals.csv"))
149     _write_report_json(video_id, segments, status, os.path.join(out_local,
150 "report.json"))
151
152     out_prefix = args.out_uri.rstrip("/")
153     pushed = []
154     for fn in sorted(os.listdir(out_local)):
155         uri = f"{out_prefix}/outputs/{video_id}/{fn}"
156         storage.put(os.path.join(out_local, fn), uri, config=storage_cfg)
157         pushed.append(uri)
158
159     print(f"[worker] infer done: {len(segments)} segment(s); pushed {len(pushed)}
160 artifact(s):")
161     for u in pushed:
162         print("    ", u)
163     return 0
164
165     #: Video extensions accepted under <corpus>/videos/ (mirrors the labels/ '.csv' filter
166 so a
167 #: per-video sidecar ? .srt/.json/thumbnail ? can't shadow the real video on a stem
168 collision).
169 _VIDEO_EXTS = (".mp4", ".mov", ".mkv", ".avi", ".webm", ".m4v")

```

```

167     def _probe_video_fps_width(path: str):
168         """Robust ffprobe (fps, frame_width) for a real video. Returns None on ANY failure.
169
170         The detector trajectory is frame-indexed; the harness maps frame->time via fps, so a
WRONG
171         fps silently mis-aligns every rally label (e.g. a 60fps clip read as 30 doubles
every time).
172         cv2's ``_probe_fps_width`` silently defaults to (30, 1280) on a read miss ? fine for
the
173         inference report (it flags the fallback) but corrupting for REAL training data. So
here we
174         probe with ffprobe (same tool as ``get_video_duration``) and return None so the
caller SKIPS
175         rather than guesses.
176         """
177         import json
178         import subprocess
179         try:
180             out = subprocess.check_output(
181                 ["ffprobe", "-v", "error", "-select_streams", "v:0",
182                  "-show_entries", "stream=r_frame_rate,width", "-of", "json", path],
183                 stderr=subprocess.DEVNULL).decode()
184             st = (json.loads(out).get("streams") or [{}])[0]
185             num, _, den = str(st.get("r_frame_rate", "0/1")).partition("/")
186             fps = float(num) / float(den or "1")
187             width = float(st.get("width") or 0)
188             if fps > 0 and width > 0:
189                 return fps, width
190         except (subprocess.SubprocessError, ValueError, KeyError, OSError,
json.JSONDecodeError):
191             pass
192         return None
193
194
195     def _resolve_train_detector(args: argparse.Namespace, config: Any):
196         """Build the trajectory DetectorRunner for ``--trajectory-source detector``, honouring
197         detector_impl (RALLY_DETECTOR_IMPL / indexing.detector_impl: auto=WSL, native=GPU
box,
198         stub=CPU mock). Returns the runner, or prints an error + returns None.
199
200         Reuses the segmenter's ``_resolve_runner`` seam so the worker and the live CLI build
the
201         detector identically (one source of truth). A non-trajectory segmenter (e.g. yolo)
has
202         no shuttle detector and is rejected.
203         """
204         from backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter
205
206         seg_cls = segmenter_registry.get(args.segmenter)
207         if not seg_cls:
208             print(f"[worker] unknown segmenter: {args.segmenter!r} "
209                   f"(have {list(segmenter_registry.list_available())})")
210             return None
211         seg = seg_cls(None, config)
212         if not isinstance(seg, TrajectoryHybridSegmenter):
213             print(f"[worker] --segmenter {args.segmenter!r} has no shuttle detector; "
214                   f"use a trajectory hybrid (wasb_hybrid / tracknet_hybrid /
fusion_hybrid).")
215             return None
216         try:
217             runner, _ = seg._resolve_runner(config.get("indexing", {}))
218         except NotImplementedError as e:
219             # e.g. detector_impl='native' for a family without a native runner (tracknet, or
220             # fusion with a tracknet substrate). Fail cleanly (rc!=0), not with a raw
traceback.

```



```

221         print(f"[worker] detector not available: {e}")
222         return None
223     ok, msg = runner.healthcheck()
224     if not ok:
225         print(f"[worker] detector environment not ready: {msg}")
226         return None
227     print(f"[worker] detector ready ({args.segmenter}): {msg}")
228     return runner
229
230
231 def cmd_train(args: argparse.Namespace) -> int:
232     """Gen-0 train-from-bucket: pull labels (+ videos) -> trajectories -> manifest ->
shell out
233     to training.gen0.harness (backend must NOT import training) -> push the artifact
bundle.
234
235     Two trajectory sources (the train pipeline + harness are identical either way):
236     * ``synth`` (default) ? the CPU "mock GPU": deterministic, label-consistent
synthetic
237         shuttle paths (no GPU/WSL), so the whole pipeline is validated on a CPU box /
CI.
238     * ``detector`` ? REAL shuttle trajectories: pull each video and run the resolved
239         DetectorRunner (native WASB on a GPU box, or stub for a GPU-free wiring test).
This
240         is the M3.5 "first real training" path; only the trajectory source flips.
241     """
242     import subprocess
243
244     config = load_config(args.config) if args.config else load_config()
245     _apply_overrides(config, args.set)
246     storage_cfg = config.storage.model_dump()
247
248     scratch = args.scratch or tempfile.mkdtemp(prefix="rally-train-")
249     labels_dir = os.path.join(scratch, "labels")
250     traj_dir = os.path.join(scratch, "trajectories")
251     videos_dir = os.path.join(scratch, "videos")
252     os.makedirs(labels_dir, exist_ok=True)
253     os.makedirs(traj_dir, exist_ok=True)
254
255     corpus = args.corpus_uri.rstrip("/")
256     label_uris = sorted(u for u in storage.list_uris(f"{corpus}/labels/",
config=storage_cfg)
257                         if u.lower().endswith(".csv"))
258     if len(label_uris) < 2:
259         print(f"[worker] need >=2 labeled videos for leave-one-video-out; found
{len(label_uris)} "
260             f"under {corpus}/labels/")
261         return 2
262
263     # Real-detector source: resolve the runner once + index the bucket's videos/ by
stem.
264     runner = None
265     video_by_stem: dict = {}
266     if args.trajectory_source == "detector":
267         os.makedirs(videos_dir, exist_ok=True)
268         runner = _resolve_train_detector(args, config)
269         if runner is None:
270             return 2
271         for vuri in storage.list_uris(f"{corpus}/videos/", config=storage_cfg):
272             vname = storage.parse_uri(vuri).name
273             if not vname.lower().endswith(_VIDEO_EXTS):
274                 continue # skip sidecars (.srt/.json/thumbnails) so they can't shadow
the video
275             video_by_stem[os.path.splitext(vname)[0]] = vuri
276

```

```

277     print(f"[worker] trajectory source: {args.trajectory_source}")
278     specs: List[dict] = []
279     for uri in label_uris:
280         fname = storage.parse_uri(uri).name # e.g.
GX020094_proxy.rallies.csv
281         name = fname.replace(".rallies.csv", "").replace(".csv", "")
282         local_label = storage.fetch(uri, os.path.join(labels_dir, fname),
config=storage_cfg)
283         if args.trajectory_source == "detector":
284             vuri = video_by_stem.get(name)
285             if not vuri:
286                 print(f"[worker] no video for {name!r} under {corpus}/videos/ ?
skipping.")
287                 continue
288                 local_video = storage.fetch(vuri, os.path.join(videos_dir,
storage.parse_uri(vuri).name),
289                                     config=storage_cfg)
290                 video_id = compute_video_id(local_video)
291                 # Real fps/width drive the trajectory frame->time mapping; PROBE (don't
guess) ? a
292                 # wrong fps silently mis-aligns every label, so skip the video if ffprobe
can't read it.
293                 meta = _probe_video_fps_width(local_video)
294                 if meta is None:
295                     print(f"[worker] could not probe fps/width for {name!r} (ffprobe) ?
skipping (won't guess).")
296                     continue
297                     fps, frame_width = meta
298                     traj = runner.run_predict(local_video, traj_dir, video_id=video_id)
299                     if not traj:
300                         print(f"[worker] detector produced no trajectory for {name!r} ?
skipping.")
301                         continue
302                         specs.append({"name": name, "trajectory": traj, "labels": local_label,
303                                     "fps": fps, "frame_width": frame_width})
304                     else:
305                         from backend.pipeline.detectors.stub_runner import
synth_trajectory_from_labels
306                         traj = os.path.join(traj_dir, f"{name}_traj.csv")
307                         synth_trajectory_from_labels(local_label, traj, fps=args.fps,
frame_width=args.frame_width)
308                         specs.append({"name": name, "trajectory": traj, "labels": local_label,
309                                     "fps": args.fps, "frame_width": args.frame_width})
310
311         if len(specs) < 2:
312             print(f"[worker] need >=2 videos with trajectories for leave-one-video-out; got
{len(specs)}.")
313             return 2
314         manifest_path = os.path.join(scratch, "manifest.json")
315         with open(manifest_path, "w", encoding="utf-8") as f:
316             json.dump(specs, f, indent=2)
317         src_label = "real detector" if args.trajectory_source == "detector" else "CPU-mock
stub"
318         print(f"[worker] built manifest: {len(specs)} videos ({src_label} trajectories)")
319
320         out_dir = os.path.join(scratch, "artifacts")
321         cmd = [sys.executable, "-m", "training.gen0.harness",
322               "--manifest", manifest_path, "--out", out_dir, "--version", args.version]
323         if args.floor:
324             floor_local = (storage.fetch(args.floor, os.path.join(scratch, "floor.json"),
config=storage_cfg)
325                             if "://" in args.floor else args.floor)
326             cmd += ["--floor", floor_local]
327         print("[worker] training:", " ".join(cmd))
328         rc = subprocess.run(cmd).returncode

```

```

329         if rc != 0:
330             print(f"[worker] gen0 harness failed (rc={rc})")
331             return rc
332
333         # PUSH the versioned artifact bundle (weights.json + manifest.json) to the bucket.
334         bundle = os.path.join(out_dir, args.version)
335         out_prefix = args.out_uri.rstrip("/")
336         pushed = []
337         for fn in sorted(os.listdir(bundle)):
338             uri = f"{out_prefix}/weights/{args.version}/{fn}"
339             storage.put(os.path.join(bundle, fn), uri, config=storage_cfg)
340             pushed.append(uri)
341         print(f"[worker] train done: pushed {len(pushed)} artifact(s):")
342         for u in pushed:
343             print("    ", u)
344         return 0
345
346
347     def build_parser() -> argparse.ArgumentParser:
348         p = argparse.ArgumentParser(prog="python -m backend.worker",
349                                     description="Storage-aware worker: pull ? run pipeline ?
350 push.")
351         sub = p.add_subparsers(dest="cmd", required=True)
352
353         pi = sub.add_parser("infer", help="run inference on one video URI and push outputs")
354         pi.add_argument("--video-uri", required=True, help="local path / local:// / s3:// /
355 gcs://")
356         pi.add_argument("--out-uri", required=True, help="output prefix
357 (outputs/<video_id>/? is appended)")
358         pi.add_argument("--segmenter", default=None, help="default:
359 indexing.default_segmenter")
360         pi.add_argument("--config", default=None, help="config.json path (default:
361 ./config.json)")
362         pi.add_argument("--scratch", default=None, help="local scratch dir (default: a temp
363 dir)")
364         pi.add_argument("--max-frames", type=int, default=None)
365         pi.add_argument("--set", action="append", default=[], metavar="k=v",
366                         help="config override, e.g. --set indexing.skip_ai_handoff=true")
367         pi.set_defaults(func=cmd_infer)
368
369         pt = sub.add_parser("train", help="Gen-0 train-from-bucket (labels [+videos] ->
370 trajectories -> harness)")
371         pt.add_argument("--corpus-uri", required=True, help="prefix containing labels/ (+
372 videos/ for "
373                     "--trajectory-source detector), e.g. s3://bucket or gcs://bucket")
374         pt.add_argument("--out-uri", required=True, help="output prefix (weights/<version>/
375 is appended)")
376         pt.add_argument("--version", required=True, help="semver for the artifact bundle")
377         pt.add_argument("--trajectory-source", choices=["synth", "detector"],
378                         default="synth",
379                         help="synth = CPU-mock label-consistent trajectories (default, no
380 GPU); "
381                             "detector = REAL shuttle trajectories via the resolved
382 DetectorRunner "
383                             "(needs videos/ in the corpus; honours detector_impl).")
384         pt.add_argument("--segmenter", default="wasb_hybrid",
385                         help="trajectory hybrid whose detector is used for --trajectory-
386 source detector")
387         pt.add_argument("--floor", default=None, help="floor baseline JSON (local path or
388 storage URI)")
389         pt.add_argument("--config", default=None)
390         pt.add_argument("--scratch", default=None)
391         pt.add_argument("--fps", type=float, default=30.0, help="synth trajectory fps
392 (detector probes the video)")
393         pt.add_argument("--frame-width", type=float, default=1280.0,

```

```

379             help="synth trajectory frame width (detector probes the video)")
380         pt.add_argument("--set", action="append", default=[], metavar="k=v")
381         pt.set_defaults(func=cmd_train)
382         return p
383
384
385     def main(argv=None) -> int:
386         args = build_parser().parse_args(argv)
387         return args.func(args)
388
389
390     if __name__ == "__main__":
391         sys.exit(main())
392

```

7. user (2026-06-20T16:58:27.119Z)

```

1     import os
2     import time
3     import logging
4     import subprocess
5     import concurrent.futures
6     from typing import List, Dict, Any, Tuple, Callable
7     from backend.pipeline.segmenters.base import BasePlaySegmenter, segmenter_registry
8     from backend.utils.video import get_video_duration, extract_video_chunk
9     from backend.sports import sport_registry
10    from backend.providers import provider_registry
11    from backend.reporting import run_signals
12
13    logger = logging.getLogger(__name__)
14
15
16    def _chunk_workdir(video_id: str) -> str:
17        """Per-video chunk staging dir under output/chunks/<video_id>.
18
19        Concurrency isolation: two runs for DIFFERENT videos must not share one chunk dir ?
20        otherwise one run finishing (and rmdir-ing the shared dir) can undo the other's
21        in-flight work. A per-video subdir means each run only ever creates/cleans its own.
22        """
23        return os.path.join("output", "chunks", video_id)
24
25
26    def _parse_time(val) -> float:
27        """Converts a timestamp value to total float seconds.
28        Handles float/int pass-through, plain numeric strings, and
29        colon-separated formats (MM:SS, HH:MM:SS). Logs warnings
30        when unexpected formats are encountered."""
31        if isinstance(val, (int, float)):
32            return float(val)
33        if isinstance(val, str):
34            val = val.strip()
35            if ":" in val:
36                # Gemini returned MM:SS or HH:MM:SS despite being asked for decimal seconds.
37                # Convert it, but warn so the issue is visible in logs.
38                parts = val.split(":")
39                parts.reverse()
40                total = 0.0
41                for i, p in enumerate(parts):
42                    try:
43                        total += float(p) * (60 ** i)
44                    except ValueError:
45                        pass
46                logger.warning(f"Timestamp '{val}' was in colon-separated format
47                (MM:SS/HH:MM:SS) instead of decimal seconds. Converted to {total:.2f}s.")
48                return total

```

```

48         try:
49             return float(val)
50         except ValueError:
51             logger.warning(f"Could not parse timestamp string '{val}' as a number.
Defaulting to 0.0.")
52             return 0.0
53             logger.warning(f"Unexpected timestamp type {type(val).__name__} for value '{val}'.
Defaulting to 0.0.")
54             return 0.0
55
56
57     class GeminiPlaySegmenter(BasePlaySegmenter):
58         @property
59         def name(self) -> str:
60             return "gemini"
61
62         @property
63         def description(self) -> str:
64             return "High-precision AI-powered semantic play segment detection using Google
Gemini Multimodal API."
65
66         def _resolve_provider(self, model_name: str):
67             """Resolve the sport profile, provider, and API key (the process_video
preamble).
68
69             Returns (sport_profile, provider) on success, or None to signal any of the
70             early-return cases (unknown sport, missing API key, unknown provider) ? the
71             caller returns [] for None.
72             """
73             # Get sport profile config
74             sport_name = self.config.get("sport", "badminton")
75             try:
76                 sport_profile = sport_registry.get(sport_name)
77             except KeyError as e:
78                 logger.error(str(e))
79                 return None
80
81             # Get AI provider and model config
82             idx_cfg = self.config.get("indexing", {})
83             provider_name = self.config.get("ai_provider", idx_cfg.get("ai_provider",
"gemini"))
84
85             # Get appropriate API key based on the provider
86             api_key_env_var = f"{provider_name.upper()}_API_KEY"
87             api_key = os.environ.get(api_key_env_var)
88             if not api_key:
89                 from backend.pipeline.segmenters._keyhint import api_key_hint
90                 logger.error(
91                     f"{api_key_env_var} environment variable is not set! "
92                     f"To run the {provider_name} segmenter, set your API key. "
93                     + api_key_hint(api_key_env_var)
94                 )
95                 return None
96
97             try:
98                 provider = provider_registry.get(provider_name, api_key=api_key,
model_name=model_name)
99             except KeyError as e:
100                 logger.error(str(e))
101                 return None
102
103             return sport_profile, provider
104
105         def _maybe_trim_video(self, video_id: str, video_path: str) -> Tuple[str,
Callable[[], None]]:

```

```

106         """Optionally trim the video to indexing.debug_duration seconds (debug feature).
107
108         Returns (video_to_upload, cleanup_temp_files) where cleanup_temp_files removes
109         any
110         temp trimmed file created here.
111         """
112         # A: Check for debug_duration to trim the video first
113         debug_duration = self.config.get("indexing", {}).get("debug_duration")
114         video_to_upload = video_path
115         is_trimmed = False
116         temp_trimmed_path = None
117
118         if debug_duration:
119             logger.info(f"DEBUG FEATURE: Trimming video to first {debug_duration}
seconds for fast processing...")
120             temp_trimmed_path = os.path.join("output", f"temp_trimmed_{video_id}.mp4")
121             os.makedirs("output", exist_ok=True)
122             if os.path.exists(temp_trimmed_path):
123                 try:
124                     os.remove(temp_trimmed_path)
125                 except Exception:
126                     pass
127
128             cmd = [
129                 "ffmpeg", "-y",
130                 "-ss", "0",
131                 "-t", str(debug_duration),
132                 "-i", video_path,
133                 "-c", "copy",
134                 temp_trimmed_path
135             ]
136             logger.debug(f"Running command: {' '.join(cmd)}")
137             try:
138                 subprocess.run(cmd, check=True, stdout=subprocess.DEVNULL,
stderr=subprocess.DEVNULL)
139                 logger.info(f"Trimmed video successfully created at:
{temp_trimmed_path}")
140                 video_to_upload = temp_trimmed_path
141                 is_trimmed = True
142             except Exception as e:
143                 logger.warning(f"Failed to trim video using ffmpeg: {e}. Falling back to
full video.")
144
145         # Helper to clean up local temp files
146         def cleanup_temp_files():
147             if is_trimmed and temp_trimmed_path and os.path.exists(temp_trimmed_path):
148                 try:
149                     os.remove(temp_trimmed_path)
150                 except Exception:
151                     pass
152
153         return video_to_upload, cleanup_temp_files
154
155     def _process_chunked(
156         self,
157         video_id: str,
158         video_to_upload: str,
159         sport_profile,
160         provider,
161         video_duration: float,

```

8. assistant (2026-06-20T16:58:30.778Z)

Now let me look at the storage layer, the segmenter base/trajectory_hybrid, and config models to ground the helper design.

9. user (2026-06-20T16:58:31.752Z)

```
1      """Pluggable storage layer: local (default) + S3/GCS first-class buckets + Google Drive.
2
3      URI-driven (see ``ref.parse_uri``). The package import is cheap ? cloud SDKs are
imported
4      lazily, only when a non-local bucket URI is actually used (install the ``storage-s3`` /
5      ``storage-gcs`` extras). The default ``local`` backend needs no third-party deps and
preserves
6      today's behaviour byte-for-byte; ``gdrive://`` likewise needs none ? it reads a locally
MOUNTED
7      Google Drive (Google Drive for Desktop) as plain files.
8
9      Worker/dispatcher entry points speak URIs via the thin helpers ``fetch`` / ``put`` /
10     ``list_uris`` / ``get_storage``; ``local://`` (and bare paths) make ``fetch`` an
identity
11     passthrough. See ``docs/DECOUPLED_COMPUTE``.
12     """
13     from __future__ import annotations
14
15     import os
16     from typing import Any, Dict, Iterator, Optional
17
18     from backend.storage.base import StorageBackend
19     from backend.storage.ref import StorageRef, StorageStat, parse_uri
20
21     __all__ = [
22         "StorageBackend", "StorageRef", "StorageStat", "parse_uri",
23         "get_storage", "fetch", "put", "list_uris",
24     ]
25
26
27     def _backend_class(name: str):
28         """Lazy-resolve a backend class so cloud SDKs aren't imported until needed."""
29         if name == "local":
30             from backend.storage.local import LocalStorage
31             return LocalStorage
32         if name == "s3":
33             from backend.storage.s3 import S3Storage
34             return S3Storage
35         if name == "gcs":
36             from backend.storage.gcs import GCSStorage
37             return GCSStorage
38         if name == "gdrive":
39             from backend.storage.gdrive import GDriveStorage
40             return GDriveStorage
41         raise ValueError(f"unknown storage backend: {name!r} (expected
local|s3|gcs|gdrive)")
42
43
44     def get_storage(backend: Optional[str] = None, *,
45                    config: Optional[Dict[str, Any]] = None, **overrides) -> StorageBackend:
46         """Construct a backend. ``backend`` (or ``config['backend']``, default 'local')
selects
47         the implementation; its per-backend settings come from ``config[backend]`` merged
with
48         ``overrides``. Settings carry NO secrets ? only endpoints/regions/paths/source-
selectors."""
49         cfg = config or {}
50         name = backend or cfg.get("backend", "local")
51         settings: Dict[str, Any] = dict(cfg.get(name, {}) or {})
52         settings.update(overrides)
53         return _backend_class(name)(**settings)
54
55
```

```

56     def fetch(uri: str, dst: "str | os.PathLike[str] | None" = None, *,
57               config: Optional[Dict[str, Any]] = None, overwrite: bool = False) -> str:
58         """Resolve ``uri`` to a local file path. For ``local`` this is an identity
passthrough
59         (no copy); for s3/gcs it downloads to ``dst``."""
60         ref = parse_uri(uri)
61         return get_storage(ref.backend, config=config).fetch_to_local(ref, dst,
overwrite=overwrite)
62
63
64     def put(local_path: "str | os.PathLike[str]", uri: str, *,
65            config: Optional[Dict[str, Any]] = None, content_type: Optional[str] = None) ->
str:
66         """Upload/copy ``local_path`` to ``uri``; return ``uri``."""
67         ref = parse_uri(uri)
68         get_storage(ref.backend, config=config).put(local_path, ref,
content_type=content_type)
69         return uri
70
71
72     def list_uris(prefix_uri: str, *, config: Optional[Dict[str, Any]] = None) ->
Iterator[str]:
73         """Yield the URI of every object under ``prefix_uri``."""
74         ref = parse_uri(prefix_uri)
75         for r in get_storage(ref.backend, config=config).list(ref):
76             yield r.uri
77

```

10. user (2026-06-20T16:58:32.817Z)

```

225
226     # --- Phase 1: env healthcheck (fail fast with a clear message) ---
227     ok, msg = runner.healthcheck()
228     if not ok:
229         logger.error("%s detector environment not ready: %s", label, msg)
230         return []
231     logger.info("%s detector env OK: %s", label, msg)
232
233     # --- Phase 2: trajectory -> candidate rally windows ---
234     # Trajectory detectors process the whole video in one pass (they carry
235     # their own frame buffering), so unlike yolo_hybrid we don't pre-chunk.
236     t_start = time.time()
237     out_dir = os.path.join("output", self.family)
238     csv_path = runner.run_predict(video_path, out_dir, video_id=video_id)
239     if not csv_path:
240         logger.error("%s produced no trajectory; aborting.", label)
241         return []
242
243     points = runner.parse_trajectory_csv(csv_path)
244     logger.info("Parsed %d trajectory points (%d visible).",
245               len(points), sum(1 for p in points if p.visible))
246
247     all_candidate_windows = runner.trajectory_to_action_windows(
248         points, fps=fps, frame_width=frame_width, chunk_start=0.0,
249         velocity_thresh=velocity_thresh, merge_gap=merge_gap,
250         min_window_duration=min_window_duration, chunk_index=0,
251         max_window_duration=max_window_duration, min_split_gap=window_min_split_gap,
252         window_hard_cap=window_hard_cap,
253         window_inactivity_end=window_inactivity_end,
254         inactivity_rally_thresh=inactivity_rally_thresh,
255         inactivity_min_density=inactivity_min_density,
256         window_blob_fallback=window_blob_fallback,
257         window_blob_factor=window_blob_factor,
258     )
259     logger.info("Phase 2 (%s) completed in %.1fs. Candidate windows: %d",

```



```

260         label, time.time() - t_start, len(all_candidate_windows))
261
262     if log_candidates:
263         for cw in all_candidate_windows:
264             logger.info(f"[{label}] rally]
265             f"({cw['end'] - cw['start']:.1f}s) |
frames={cw['active_frame_count']} "
266             f"peak_v={cw['peak_velocity']:.3f}
mean_v={cw['mean_velocity']:.3f}")
267
268     # Per-window stats via the enrichment hook ? base = the 4 motion keys; the
fusion
269     # segmenter adds net_crossings + person-cue features. Computed once, reused for
both
270     # persistence and the handoff gate below.
271     window_stats = [
272         self._enrich_candidate_stats(cw, points, fps, frame_width, video_path,
idx_cfg)
273         for cw in all_candidate_windows
274     ]
275
276     # Persist raw candidates for the fine-tuning dataset (same table as YOLO).
277     candidate_ids: List[Optional[int]] = [None] * len(all_candidate_windows)
278     if store_candidates:
279         for i, cw in enumerate(all_candidate_windows):
280             candidate_ids[i] = self.db.add_candidate_segment(
281                 video_id, detector=self.name,
282                 start_time=cw["start"], end_time=cw["end"],
283                 chunk_index=cw["chunk_index"], confidence=min(1.0,
cw["peak_velocity"]),
284                 stats=window_stats[i], detection_params=detection_params,
285             )
286
287     if not all_candidate_windows:
288         return []
289
290     # --- Phase 3: AI provider handoff over padded candidate clips ---
291     # Which windows reach the handoff (base = all; fusion may apply Gate A/B).
Persisted
292     # candidates above stay the full superset ? only the served play_segments are
gated.
293     handoff_indices = self._select_for_handoff(all_candidate_windows, window_stats,
idx_cfg)
294
295     ai_segments: List[dict] = []
296     if skip_ai:
297         # Fully-local: the selected candidate windows ARE the rallies ? emit them
verbatim
298         # (no clip extraction, no upload). Boundaries are the local detector's; shot
count
299         # falls back to the duration-based estimate (no AI exchange count).
300         ai_segments = [
301             {
302                 "start_time": all_candidate_windows[wi]["start"],
303                 "end_time": all_candidate_windows[wi]["end"],
304                 "shuttle_exchange_count": None,
305                 "shot_count_confidence": "LocalNoAI",
306                 "_candidate_id": candidate_ids[wi],
307             }
308             for wi in handoff_indices
309         ]
310         logger.info("Phase 3 SKIPPED (fully-local): emitted %d candidate windows as
rallies "
311                     "(no AI handoff).", len(ai_segments))

```

```

312         else:
313             handoff_dir = os.path.join("output", f"chunks_{provider_name}_handoff")
314             os.makedirs(handoff_dir, exist_ok=True)
315             logger.info("Phase 3: %s validation on %d candidate windows...",
316                         provider_name, len(handoff_indices))
317
318             def process_candidate_window(wi: int, window: Dict[str, Any]) -> List[dict]:
319                 w_start = max(0, window["start"] - handoff_padding)
320                 w_end = min(video_duration, window["end"] + handoff_padding)
321                 w_dur = w_end - w_start
322                 w_path = os.path.join(handoff_dir, f"handoff_{video_id}_{wi}.mp4")
323                 if not extract_video_chunk(video_path, w_start, w_dur, w_path,
reencode=True):
324                     return []
325                 prompt = sport_profile.get_prompt(chunk_duration=w_dur)
326                 segments, _ = provider.analyze_video(w_path, prompt,
327                                                     label=f"Handoff {wi+1}")
328                 valid = []
329                 for seg in segments:
330                     try:
331                         seg["start_time"] = float(seg["start_time"]) + w_start
332                         seg["end_time"] = float(seg["end_time"]) + w_start
333                         seg["_candidate_id"] = candidate_ids[wi]
334                         valid.append(seg)
335                     except (ValueError, TypeError, KeyError) as e:
336                         logger.warning("Dropping segment with unparseable timestamps: %s
? %s", seg, e)
337                     try:
338                         os.remove(w_path)
339                     except OSError:
340                         pass
341                 return valid
342
343             with concurrent.futures.ThreadPoolExecutor(max_workers=ai_max_workers) as
executor:
344                 futures = [executor.submit(process_candidate_window, wi,
all_candidate_windows[wi])
345                             for wi in handoff_indices]
346                 for future in futures:
347                     ai_segments.extend(future.result())
348                 try:
349                     os.rmdir(handoff_dir)
350                 except OSError:
351                     pass
352                 logger.info("Phase 3 completed. AI confirmed %d semantic play segments.",
len(ai_segments))
353
354             # --- Phase 4: DB population + candidate linking ---

```

11. assistant (2026-06-20T16:58:36.823Z)

Now let me check the config models for OutputConfig/StorageConfig and the yolo_hybrid hardcoded paths, plus the detector runner signature for `run\_predict`.

12. user (2026-06-20T16:58:38.863Z)

```

220         if self.chunk_overlap >= self.chunk_duration:
221             raise ValueError(
222                 f"indexing.chunk_overlap ({self.chunk_overlap}) must be < chunk_duration
"
223                 f"({self.chunk_duration}); otherwise chunking never advances."
224             )
225         return self
226

```

```

227
228 class OutputConfig(BaseModel):
229     default_highlights_name: str = "highlights.mp4"
230     db_name: str = "sports_indexer.db"
231     # Directory where the DB, highlight reels, and processed clips are written.
232     # The CLI's --output-dir overrides this at startup (see backend/main.py::main).
233     output_dir: str = "output"
234
235
236 class WatermarkConfig(BaseModel):
237     """Branding overlay burned into each highlight clip during the per-clip re-encode
238     (backend/pipeline/stitcher/compiler.py). **On by default** ? set `enabled: false`
239     (under `stitching.watermark` in config.json) to turn it off. The text is fully
240     configurable. The concat stage stays stream-copy (-c copy)."""
241     enabled: bool = True
242     text: str = "Generated by Khelsutra.guru"
243     logo_path: str = "" # optional transparent PNG overlay; "" = no logo
244     font_path: str = "" # optional .ttf; "" = use the fontconfig `font` family
below
245     font: str = "Sans" # fontconfig family used when font_path is empty
246     font_size_ratio: float = Field(default=0.035, gt=0.0, le=0.5) # text height as a
fraction of frame height
247     opacity: float = Field(default=0.85, ge=0.0, le=1.0)
248     box: bool = True # faint backing box behind the text for legibility
249     margin: int = Field(default=24, ge=0) # px inset from the chosen corner
250     position: Literal["bottom-right", "bottom-left", "top-right", "top-left"] = "bottom-
right"
251
252
253 class StitchingConfig(BaseModel):
254     begin_padding: float = 0.5
255     end_padding: float = 0.5
256     use_cache: bool = False
257     min_shot_count: int = 1
258     # Long-rally reel filter (seconds; 0 = OFF). Drop detected rally pairs whose
duration
259     # (end ? start) is below this from the highlight reel, BEFORE compiling. A cleaner
260     # reel-selection knob than `min_shot_count`: shot_count is unlabeled / "Unknown" on
the
261     # fully-local no-AI path, whereas duration is derived from the rally's own
start/end. The
262     # local detector over-fires and its false positives are mostly SHORT (motion blips,
263     # background-court fragments), so this keeps the reel to the eye-catching long
rallies.
264     # OFF by default ? the detector output is unchanged, only which rallies reach the
reel.
265     min_rally_duration: float = Field(default=0.0, ge=0.0)
266     watermark: WatermarkConfig = Field(default_factory=WatermarkConfig)
267
268
269 class LoggingConfig(BaseModel):
270     """Logging knobs for the run entrypoints (see backend/utils/logging_setup.py)."""
271     # Third-party HTTP/RPC client loggers (httpx, httpcore, urllib3, google-genai,
272     # google.auth, grpc) emit one INFO line per request ? dozens per Gemini run, which
273     # buries our own [rally]/[compile] lines in run.log during manual QA. Default raises
274     # them to WARNING; set true to restore full chatter for request-flow debugging.
275     verbose_http: bool = False
276
277
278 class SecurityConfig(BaseModel):
279     # When set, /api/process video_path must resolve under this directory (hosted/tenant
280     # mode). Default None preserves the single-user local 'any local file' workflow.
281     allowed_video_root: Optional[str] = None
282
283     # --- Chunked upload (B2, PR-2) ---

```

```

284 # Landing zone for browser uploads (assembled from parts). Per-user subdirectories
285 # arrive with PR-3 identity scoping. ? In hosted mode, set allowed_video_root to
286 # contain upload_dir or /api/process will reject the uploaded paths.
287 upload_dir: str = "output/uploads"
288 # Whole-file cap (default 4 GB) and per-part cap. Parts stay under ~100 MB because
289 # free-tier edge proxies cap request bodies there (HOSTING_PLAN $2).
290 max_upload_mb: int = 4096
291 max_part_mb: int = 95
292 allowed_upload_extensions: List[str] = ["mp4", "mov"]
293
294
295 class StorageConfig(BaseModel):
296     """Pluggable storage backend (docs/DECOUPLED_COMPUTE/). Default ``local`` = today's
297     behaviour (filesystem paths, byte-for-byte). ``s3`` covers AWS + Cloudflare R2 + DO
Spaces
298     + MinIO (via ``endpoint_url``); ``gcs`` = Google Cloud Storage; ``gdrive`` = a
locally MOUNTED
299     Google Drive (``gdrive.mount_root``). Per-backend settings are loose dicts passed to
the constructor ?
300     **secrets are never stored here**, only endpoints / regions / file paths / source-
selectors
301     (boto3/google auth chains supply credentials). See ``backend/storage/``. """
302     backend: Literal["local", "s3", "gcs", "gdrive"] = "local"
303     # Output root for produced artifacts. "" => fall back to output.output_dir (non-
breaking).
304     output_root: str = ""
305     local: dict[str, Any] = Field(default_factory=dict)
306     s3: dict[str, Any] = Field(default_factory=dict)
307     gcs: dict[str, Any] = Field(default_factory=dict)
308     gdrive: dict[str, Any] = Field(default_factory=dict)
309
310
311 class AppConfig(BaseModel):
312     """Root configuration object.
313
314     All runtime configuration should be accessed via an instance of this class
315     (or the loader) rather than raw dict lookups.
316     """
317
318     sport: Sport = "badminton"
319     ai_provider: str = "gemini"
320     # Auto-tracking alias: 2.5-flash is legacy now that 3.5-flash is the stable
flagship;
321     # the alias never deprecates underneath us. Eval reports record the run date
alongside.
322     ai_model: str = "gemini-flash-latest"
323
324     validation: ValidationConfig = Field(default_factory=ValidationConfig)
325     indexing: IndexingConfig = Field(default_factory=IndexingConfig)
326     output: OutputConfig = Field(default_factory=OutputConfig)
327     stitching: StitchingConfig = Field(default_factory=StitchingConfig)
328     security: SecurityConfig = Field(default_factory=SecurityConfig)
329     logging: LoggingConfig = Field(default_factory=LoggingConfig)
330     storage: StorageConfig = Field(default_factory=StorageConfig)
331
332     # Allow extra keys during transition so existing config.json files do not break.
333     model_config = {
334         "extra": "allow",
335         "validate_assignment": True,
336     }
337
338     # Temporary migration shim for legacy dict-style access (config.get("section",
{ }).get(...)).
339     # Will be removed once all call sites (validators, segmenters, etc.) move to typed

```

13. user (2026-06-20T16:58:39.179Z)

```
240         if store_candidates:
241             for i, w in enumerate(all_candidate_windows):
242                 stats = {
243                     "peak_velocity": w["peak_velocity"],
244                     "mean_velocity": w["mean_velocity"],
245                     "active_frame_count": w["active_frame_count"],
246                     "track_id_count": w["track_id_count"],
247                 }
248                 # Use peak velocity (capped at 1.0) as a coarse generic confidence
score.
249                 confidence = min(1.0, w["peak_velocity"])
250                 candidate_ids[i] = self.db.add_candidate_segment(
251                     video_id, detector="yolo_hybrid",
252                     start_time=w["start"], end_time=w["end"],
253                     chunk_index=w["chunk_index"], confidence=confidence,
254                     stats=stats, detection_params=detection_params,
255                 )
256
257             return candidate_ids
258
259     def _run_ai_handoff(
260         self, video_id: str, video_path: str, video_duration: float,
261         all_candidate_windows: List[Dict[str, Any]], candidate_ids: List[Any],
262         provider, sport_profile, provider_name: str, chunk_duration: float,
263         handoff_padding: float, ai_max_workers: int,
264     ) -> List[dict]:
265         """Phase 3: AI Provider Handoff ? re-encode each padded candidate window and ask
266         the provider for semantic boundaries, in parallel. Verbatim move."""
267         gemini_segments = []
268         gemini_dir = os.path.join("output", f"chunks_{provider_name}_handoff")
269         os.makedirs(gemini_dir, exist_ok=True)
270
271         logger.info(f"Starting Phase 3: AI Provider ({provider_name}) Validation on
{len(all_candidate_windows)} candidate windows...")
272
273         def process_candidate_window(wi: int, window: Dict[str, Any]) -> List[dict]:
274             w_start = max(0, window["start"] - handoff_padding)
275             w_end = min(video_duration, window["end"] + handoff_padding)
276             w_dur = w_end - w_start
277
278             w_path = os.path.join(gemini_dir, f"handoff_{video_id}_{wi}.mp4")
279             # Re-encode short handoff clips to avoid keyframe-boundary drift
280             success = extract_video_chunk(video_path, w_start, w_dur, w_path,
reencode=True)
281             if not success:
282                 return []
283
284             prompt = sport_profile.get_prompt(chunk_duration=w_dur)
285             segments, metrics = provider.analyze_video(w_path, prompt,
chunk_duration=w_dur, label=f"Handoff {wi+1}")
286
287             # Coordinate mapping ? shift local timestamps back to full-video time
288             valid_segments = []
289             for segment in segments:
290                 try:
291                     segment["start_time"] = float(segment["start_time"]) + w_start
292                     segment["end_time"] = float(segment["end_time"]) + w_start
293                     # Stamp the source candidate so Phase 4 can link
prediction->confirmation.
294                     segment["_candidate_id"] = candidate_ids[wi]
295                     valid_segments.append(segment)
296                 except (ValueError, TypeError, KeyError) as e:
297                     logger.warning(f"Dropping segment with unparseable timestamps:
```

```

{segment} ? {e}")
298
299         try:
300             os.remove(w_path)
301         except OSError:
302             pass
303
304         return valid_segments
305
306         with concurrent.futures.ThreadPoolExecutor(max_workers=ai_max_workers) as
executor:
307             futures = [executor.submit(process_candidate_window, wi, w) for wi, w in
enumerate(all_candidate_windows)]
308             # Iterate in submission order to preserve temporal ordering
309             for future in futures:

```

14. assistant (2026-06-20T16:58:43.012Z)

Now let me check the detector runner signatures (run_predict) and how main.py / database.py set output_dir, plus the yolo chunks_dir and the eval defaults, to ground the exact line numbers.

15. user (2026-06-20T16:58:44.462Z)

```

80         return False, "`wsl` not found ? Windows + WSL2 required."
81     except subprocess.TimeoutExpired:
82         return False, "WSL healthcheck timed out."
83     out = (res.stdout or "").strip()
84     if res.returncode != 0:
85         return False, f"WSL `wasb` env check failed: {(res.stderr or out).strip()}"
86     return True, out
87
88     def _stage_video(self, video_win_path: str, dest_name: str) -> Optional[str]:
89         src_mnt = to_wsl_mnt_path(video_win_path)
90         reason = self.wsl_source_unreadable_reason(src_mnt)
91         if reason:
92             logger.error("Cannot stage video into WSL: %s", reason)
93             return None
94         dest = f"{self.cfg.wsl_stage_dir}/{dest_name}"
95         cmd = f"mkdir -p {self.cfg.wsl_stage_dir} && cp {shlex.quote(src_mnt)}
{wsl_tilde_quote(dest)} && echo OK"
96         try:
97             res = self._wsl_bash(cmd)
98         except subprocess.TimeoutExpired:
99             logger.error("Staging video into WSL timed out.")
100             return None
101         if res.returncode != 0 or "OK" not in (res.stdout or ""):
102             logger.error("Failed to stage video into WSL: %s", (res.stderr or
res.stdout).strip())
103             return None
104         return dest
105
106     def _stage_infer_script(self) -> bool:
107         """Copy wasb_infer.py into the WASB repo src/ so it can import WASB modules."""
108         src_mnt = to_wsl_mnt_path(_INFER_WIN)
109         cmd = f"cp {shlex.quote(src_mnt)} {self.cfg.repo_dir}/src/wasb_infer.py && echo
OK"
110         res = self._wsl_bash(cmd)
111         return res.returncode == 0 and "OK" in (res.stdout or "")
112
113     def run_predict(self, video_win_path: str, output_win_dir: str,
114                    video_id: Optional[str] = None) -> Optional[str]:
115         """Run WASB on a video. Returns the Windows path to the trajectory CSV, or None.
116
117         All WSL/cache/output artifacts are namespaced by ``video_id`` (the file MD5) so
118         two videos that share a filename (e.g. two ``match.mp4`` from different folders)

```

```

119         can never reuse each other's staged frames, resumable cache, or CSV.
120         """
121         # Resolve relative dirs (the hybrids pass "output/wasb") BEFORE the /mnt
122         # translation: to_wsl_mnt_path passes relative paths through unchanged, so
123         # WSL resolved them against the inference cwd (~/models/WASB-SBDT/src) and
124         # the CSV landed inside WSL while Windows polled the repo-relative path.
125         output_win_dir = os.path.abspath(output_win_dir)
126         os.makedirs(output_win_dir, exist_ok=True)
127         # Normalize for cross-platform basename (tests use Windows paths on Linux).
128         video_win_path = str(video_win_path).replace("\\", "/")
129         base = os.path.basename(video_win_path)
130         stem, ext = os.path.splitext(base)
131         # Unique, content-derived key: same filename + different bytes -> different key.
132         key = f"{stem}__{video_id[:12]}" if video_id else stem
133         wsl_out_dir = to_wsl_mnt_path(output_win_dir)
134         expected_csv_win = os.path.join(output_win_dir, f"{key}_wasb.csv")
135
136         if not self._stage_infer_script():
137             logger.error("Could not stage wasb_infer.py into WSL.")
138             return None
139         staged_video = self._stage_video(video_win_path, f"{key}{ext}")
140         if staged_video is None:
141             return None
142         frames_dir = f"{self.cfg.wsl_stage_dir}/{key}_frames"
143
144         # Disk guard: a 4K clip can extract ~100 GB of PNG frames. Warn early if the
145         # WSL stage filesystem is already low so the user can clean up before it fills.
146         # (Streaming never materializes the PNGs, so the guard only matters off the fast
147         path.)
148         if self.cfg.min_free_gb > 0 and not self.cfg.stream_video:
149             free = self._wsl_free_gb(self.cfg.wsl_stage_dir)
150             if free is not None and free < self.cfg.min_free_gb:
151                 logger.warning(
152                     "Low WSL disk: %.1f GB free at %s (< min_free_gb=%.1f). Frame
153 extraction "
154                     "may fill the disk ? consider running tools/cleanup_caches.py.",
155                     free, self.cfg.wsl_stage_dir, self.cfg.min_free_gb)
156
157         # Fast path: stream-decode the video in-process (no PNG extraction). Output is
158         # bit-identical to the PNG round-trip but avoids writing ~46k PNGs for a 12-min
159         clip.
160         if self.cfg.stream_video:
161             frame_args = "--stream-video "
162         else:
163             frame_args = f"--frames_out_dir {wsl_tilde_quote(frames_dir)} "
164         cmd = (
165             f"conda activate {self.cfg.conda_env} && cd {self.cfg.repo_dir}/src && "
166             f"python wasb_infer.py "
167             f"--video {wsl_tilde_quote(staged_video)} "
168             f"{frame_args}"
169             f"--weights {wsl_tilde_quote(self.cfg.weights_path)} "
170             f"--sport {shlex.quote(self.cfg.sport)} "
171             f"--out {wsl_tilde_quote(wsl_out_dir + '/' + key + '_wasb.csv')}"
172         )

```

16. user (2026-06-20T16:58:45.414Z)

```

395         # Snapshot of the params that produced these detections ? stored alongside each
396         # candidate so a fine-tuning run can reproduce / correlate against them.
397         detection_params = {
398             "velocity_thresh": velocity_thresh,
399             "merge_gap": merge_gap,
400             "frame_skip": frame_skip,
401             "tracker": tracker_config,
402             "detector_model": idx_cfg.get("detector_model", _DEFAULT_DETECTOR),

```

```

403         "detector_score_threshold": idx_cfg.get("detector_score_threshold", 0.5),
404         "min_action_window_duration": idx_cfg.get("min_action_window_duration",
2.0),
405     }
406
407     chunks_dir = os.path.join("output", "chunks_yolo")
408     os.makedirs(chunks_dir, exist_ok=True)
409
410     # Phase 2: chunk ? per-chunk action windows ? cross-chunk overlap merge.
411     all_candidate_windows = self._collect_candidate_windows(
412         video_id, video_path, video_duration, idx_cfg,
413         chunk_duration, chunk_overlap, velocity_thresh, merge_gap,
414         frame_skip, chunks_dir,
415     )
416
417     # Phase 2b: log + persist raw candidates (window index -> candidate_id).
418     candidate_ids = self._persist_candidates(
419         video_id, all_candidate_windows, detection_params, log_candidates,
store_candidates,
420     )
421
422     try:
423         os.rmdir(chunks_dir)
424     except Exception:
425         pass
426
427     # Phase 3: AI Provider Handoff
428     gemini_segments = self._run_ai_handoff(
429         video_id, video_path, video_duration, all_candidate_windows, candidate_ids,

```

17. assistant (2026-06-20T16:58:49.796Z)

Now let me check how `main.py` sets the output_dir (to understand the live CLI path) and how the segmenter base accesses config so the helper can read `output\_root`/`output\_dir`. Also check the compiler output path and the reporting harvest write.

18. user (2026-06-20T16:58:51.823Z)

```

229:         # report.output_dir). Never raises. Surface the paths in the response.
310:     (which only writes into output_dir; the browser previously got back an unreachable
318:     output_dir = getattr(getattr(global_config, "output", None), "output_dir", "output") or
"output"
319:     path = os.path.join(output_dir, name)
321:     path = resolve_under_root(path, output_dir)
389:     output_dir = getattr(getattr(global_config, "output", None), "output_dir", "output") or
"output"
390:     compiler = VideoCompiler(output_dir, begin_padding=stitching.begin_padding,
514:         out_dir = args.output_dir
546:     # on the typed model (OutputConfig.output_dir), so every consumer ? including
548:     global_config.output.output_dir = args.output_dir
549:     os.makedirs(global_config.output.output_dir, exist_ok=True)
551:     db_path = os.path.join(global_config.output.output_dir, global_config.output.db_name)

```

19. user (2026-06-20T16:58:53.242Z)

```

72:         output_dir = (config.get("report") or {}).get("output_dir")
73:         res = rec.write(output_dir=output_dir)

```

20. user (2026-06-20T16:58:53.880Z)

```

1     from abc import ABC, abstractmethod
2     from typing import List, Dict, Any, Type, Optional
3     from backend.infrastructure.database import Database
4     from backend.results import Result # new standardized contract (Result[Value] ==

```



```

Success[Value] | Failure)
5
6     class BasePlaySegmenter(ABC):
7         def __init__(self, db: Database, config: Dict[str, Any]):
8             self.db = db
9             self.config = config
10
11         @property
12         @abstractmethod
13         def name(self) -> str:
14             """Unique identifier for this segmenter."""
15             pass
16
17         @property
18         @abstractmethod
19         def description(self) -> str:
20             """Human-readable explanation of this segmenter."""
21             pass
22
23         @abstractmethod
24         def process_video(self, video_id: str, video_path: str, player_pool: List[str] =
None, max_frames: int = None) -> "Result[List[Dict[str, Any]]]":
25             """
26             Processes a video to detect play segments, populates SQLite database,
27             and returns a Result.
28
29             Use isinstance(result, Failure) (or result.is_success()) to distinguish
30             "no segments found" from "something went wrong" (missing key, healthcheck
31             failure, etc.). This is the start of the standardized error/result contract.
32             """
33             pass
34
35     class SegmenterRegistry:
36         def __init__(self):
37             self._segmenters: Dict[str, Type[BasePlaySegmenter]] = {}
38
39         def register(self, segmenter_cls: Type[BasePlaySegmenter]):
40             """Registers a new segmenter class under its name property."""
41             try:
42                 temp_inst = segmenter_cls(None, {})
43                 name = temp_inst.name
44             except Exception as e:
45                 raise ValueError(f"Failed to resolve name property from segmenter class
{segmenter_cls.__name__}: {e}") from e
46             self._segmenters[name] = segmenter_cls
47
48         def get(self, name: str) -> Optional[Type[BasePlaySegmenter]]:
49             return self._segmenters.get(name)
50
51         def list_available(self) -> Dict[str, str]:
52             """Returns a dict of registered segmenter names and their descriptions."""
53             res = {}
54             for name, cls in self._segmenters.items():
55                 try:
56                     temp_inst = cls(None, {})
57                     desc = temp_inst.description
58                 except Exception:
59                     desc = "No description available."
60             res[name] = desc

```

21. assistant (2026-06-20T16:58:58.457Z)

I have enough grounding. Let me verify the database `add\_video` signature and check whether there is already a workspace-like helper or `output\_root` consumer, plus the compiler output and the config `.get` shim that segmenters use.

22. user (2026-06-20T16:59:01.155Z)

```
287:     def add_video(self, video_id: str, filepath: str, status: str, validation_results:
List[Dict[str, Any]]) -> bool:
```

23. user (2026-06-20T16:59:01.632Z)

No files found

24. user (2026-06-20T16:59:02.263Z)

Found 8 files

```
backend\config\models.py
backend\storage\gdrive.py
backend\storage\base.py
backend\storage\__init__.py
backend\worker\__main__.py
backend\storage\s3.py
backend\storage\local.py
backend\storage\gcs.py
```

25. assistant (2026-06-20T16:59:07.299Z)

I have confirmed: `output\_root` is defined in config but consumed only in storage's own modules (not yet wired into the writers), there is no existing `workspace.py`, and `add\_video` takes no workspace reference. The design needs to add a workspace_uri column. Let me verify the videos table schema and migration ladder briefly to know the latest user_version.

26. user (2026-06-20T16:59:08.299Z)

```
100             if version < 4:
101                 self._migrate_to_v4(cursor)
102
103             if version < 5:
104                 self._migrate_to_v5(cursor)
105             # future: if version < 6: cursor.execute("ALTER TABLE ..."); PRAGMA
user_version = 6
106
107             conn.commit()
108
109     def _create_base_tables(self, cursor):
110         """Create the videos / play_segments / events base tables (idempotent)."""
111         # Videos Table
112         cursor.execute("""
113             CREATE TABLE IF NOT EXISTS videos (
114                 id TEXT PRIMARY KEY,
115                 filepath TEXT NOT NULL,
116                 processed_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
117                 status TEXT NOT NULL,
118                 validation_results TEXT
119             )
120         """)
121
122         # Play Segments Table (Extensible metadata JSON)
123         cursor.execute("""
124             CREATE TABLE IF NOT EXISTS play_segments (
125                 id INTEGER PRIMARY KEY AUTOINCREMENT,
126                 video_id TEXT NOT NULL,
127                 start_time REAL NOT NULL,
128                 end_time REAL NOT NULL,
129                 duration REAL NOT NULL,
130                 server TEXT,
131                 receiver TEXT,
132                 metadata TEXT,
```

```

133         FOREIGN KEY (video_id) REFERENCES videos(id) ON DELETE CASCADE
134     )
135     """)
136
137 # Events Table
138 cursor.execute("""
139     CREATE TABLE IF NOT EXISTS events (
140         id INTEGER PRIMARY KEY AUTOINCREMENT,
141         segment_id INTEGER NOT NULL,
142         timestamp REAL NOT NULL,
143         type TEXT NOT NULL,
144         player TEXT,
145         details TEXT,
146         FOREIGN KEY (segment_id) REFERENCES play_segments(id) ON DELETE CASCADE
147     )
148     """)
149
150 def _migrate_to_v1(self, cursor):
151     # v1: raw candidate detections (pre-confirmation), detector-agnostic.
152     # Common fields are columns; detector-specific metrics go in `stats` JSON,
153     # so a new segmenter reuses this table by setting its own `detector`/`stats`.
154     cursor.execute("""
155         CREATE TABLE IF NOT EXISTS candidate_segments (
156             id INTEGER PRIMARY KEY AUTOINCREMENT,
157             video_id TEXT NOT NULL,
158             detector TEXT NOT NULL,
159             chunk_index INTEGER,
160             start_time REAL NOT NULL,
161             end_time REAL NOT NULL,
162             duration REAL NOT NULL,
163             confidence REAL,
164             stats TEXT,
165             detection_params TEXT,
166             confirmed_segment_id INTEGER,
167             human_label TEXT,
168             notes TEXT,
169             created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
170             FOREIGN KEY (video_id) REFERENCES videos(id) ON DELETE CASCADE,
171             FOREIGN KEY (confirmed_segment_id) REFERENCES play_segments(id) ON
DELETE SET NULL
172         )
173         """)
174     cursor.execute("""
175         CREATE INDEX IF NOT EXISTS idx_candidates_video_detector
176             ON candidate_segments(video_id, detector)
177         """)
178     cursor.execute("PRAGMA user_version = 1")
179
180 def _migrate_to_v2(self, cursor):
181     # v2: async job model (DEFERRED_HARDENING_PLAN #16). One row per submitted
182     # /api/process request. `status` is the EXECUTION state of the job
183     # (queued|running|done|failed); the pipeline's own verdict (e.g. a video
184     # that failed validation) lives inside `result` JSON as result["status"].
185     # `result` is the full sync-era response dict (incl. report paths), so the
186     # observability report is the job's artifact, per the plan.
187     cursor.execute("""
188         CREATE TABLE IF NOT EXISTS jobs (
189             id TEXT PRIMARY KEY,
190             video_path TEXT NOT NULL,
191             video_id TEXT,
192             segmenter TEXT,
193             player_pool TEXT,
194             max_frames INTEGER,
195             status TEXT NOT NULL DEFAULT 'queued',
196             progress TEXT,

```

```

197         error TEXT,
198         result TEXT,
199         created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
200         started_at TIMESTAMP,
201         finished_at TIMESTAMP
202     )
203     """
204     cursor.execute("CREATE INDEX IF NOT EXISTS idx_jobs_status ON jobs(status)")
205     cursor.execute("PRAGMA user_version = 2")
206
207     def _migrate_to_v3(self, cursor):
208         # v3: self-scheduling execution policy (EXECUTION_POLICY.md P2). `policy` = the
209         # JSON ExecutionPolicy a job carries; `next_poll_at` = when a 'waiting' job is
210         # due
211         # to resume. A 'waiting' status + due-time lets a job SELF-SCHEDULE ? any worker
212         # re-claims it via `claim_due_job` when due ? with NO master node: the table IS
213         # the coordinator. ALTER (not recreate) so existing v2 job rows survive; the
214         # ADD COLUMNS are applied idempotently so an interrupted upgrade can re-run
215         # them.
216         self._add_column_idempotent(cursor, "ALTER TABLE jobs ADD COLUMN policy TEXT")
217         self._add_column_idempotent(cursor, "ALTER TABLE jobs ADD COLUMN next_poll_at
TIMESTAMP")
218         cursor.execute("CREATE INDEX IF NOT EXISTS idx_jobs_due ON jobs(status,
next_poll_at)")
219         cursor.execute("PRAGMA user_version = 3")
220
221     def _migrate_to_v4(self, cursor):
222         # v4: personalization groundwork (PERSONALIZATION_PLAN.md §2). Add a NULLABLE
223         # `user_id` to videos + candidate_segments so a future per-user / multi-tenant
224         # path can scope rows to an owner. **NULL = local install = byte-identical to
225         # today** ? every existing row stays NULL and every existing query (which never
226         # filters on user_id) is unaffected. ALTER (not recreate) so v1-v3 rows survive;
227         # the ADD COLUMNS are applied idempotently so an interrupted upgrade can re-run
228         # them.
229         # Additive ONLY: no served-behavior change rides on this slice.
230         self._add_column_idempotent(cursor, "ALTER TABLE videos ADD COLUMN user_id
TEXT")
231         self._add_column_idempotent(cursor, "ALTER TABLE candidate_segments ADD COLUMN
user_id TEXT")
232         # Partial indexes (WHERE user_id IS NOT NULL) keep the NULL=local fast path free

```

27. assistant (2026-06-20T16:59:13.629Z)

I now have everything I need. The latest migration is v5, so a workspace column lands as v6 (additive, NULLABLE ? back-compat safe). I have confirmed all the load-bearing facts: hardcoded `output/` paths, the `run\_predict(video\_path, out\_dir, video\_id)` seam, `output\_root` defined-but-unwired, no existing workspace helper, and `add\_video` signature. Let me produce the design.

28. user (2026-06-20T17:00:21.444Z)

Output does not match required schema: root: must have required property 'layout_tree', root: must have required property 'code_changes'

29. assistant (2026-06-20T17:00:26.398Z)

The validator rejected the call ? likely because the empty `code\_changes: []` and the multi-line `layout\_tree` string interacted with a parse issue (the `[]` literal in code_changes plus an unescaped issue). The schema requires `code\_changes` to be a populated list of objects with file/change. Let me resubmit with proper code_changes entries (file:line -> change) and keep the rest.

30. user (2026-06-20T17:01:32.330Z)

Output does not match required schema: root: must have required property 'layout_tree', root:

must have required property 'code_changes', root: must have required property 'phases'

31. assistant (2026-06-20T17:01:38.816Z)

The error is odd ? it lists ``layout_tree``, ``code_changes``, ``phases`` as missing even though I'm providing them. This suggests the JSON payload failed to parse (likely the unescaped angle brackets `<` `>` or the multi-line content broke the tool-call JSON), so the parser saw an empty/invalid object and reported all required fields missing. Let me simplify the strings (avoid `<...>` raw, avoid newlines where risky, no leading `` `` literals) and resubmit compactly.

32. user (2026-06-20T17:02:35.831Z)

Output does not match required schema: root: must have required property 'layout_tree', root: must have required property 'code_changes', root: must have required property 'phases'

33. assistant (2026-06-20T17:02:41.657Z)

The same three fields are flagged as missing despite being present. This means the JSON for the whole arguments object failed to parse, so the validator saw an empty object. The likely culprit is a stray `</invoke>` / `</parameter>` fragment I accidentally appended at the very end of the last ``phases``... no ? that was in the open_questions block. Actually the trailing `</invoke>` I typed inside the last parameter value broke the tool XML. Let me resubmit cleanly with no stray tags and shorter values to reduce parse risk.

34. user (2026-06-20T17:03:29.240Z)

Output does not match required schema: root: must have required property 'layout_tree', root: must have required property 'code_changes', root: must have required property 'phases'

35. assistant (2026-06-20T17:03:33.807Z)

I keep accidentally appending stray `</invoke>` / `</parameter>` closing tags inside the last parameter, which corrupts the tool-call XML so nothing parses. Let me submit a minimal version first to confirm the schema accepts it, with no stray tags and all required fields present.

36. user (2026-06-20T17:03:58.676Z)

Output does not match required schema: root: must have required property 'layout_tree', root: must have required property 'code_changes', root: must have required property 'phases'

37. assistant (2026-06-20T17:04:03.692Z)

I notice I keep typing a stray `</invoke>` and `</parameter>` AFTER the last real parameter, before the function's own closing tag. That extra closing tag corrupts the XML. Let me write the call with NO trailing stray tags at all ? ending the last parameter cleanly and going straight to the function close.

38. user (2026-06-20T17:04:25.404Z)

Output does not match required schema: root: must have required property 'layout_tree', root: must have required property 'code_changes', root: must have required property 'phases'

39. assistant (2026-06-20T17:04:38.439Z)

The persistent failure on exactly these three fields, despite them being present, strongly suggests the issue is structural in how the arguments object is being assembled ? the three array/required fields aren't landing at the top level. Given ``code_changes`` is an array of objects and ``phases`` an array of strings and ``layout_tree`` a string, and they keep being reported missing together, the most likely cause is that my trailing stray `</parameter>` after `open_questions_for_owner` is closing the whole arguments block early. Let me reorder so the three required-and-failing fields come FIRST, and end cleanly.

40. user (2026-06-20T17:04:59.396Z)

Output does not match required schema: root: must have required property 'convention_summary', root: must have required property 'code_changes', root: must have required property 'phases'

41. assistant (2026-06-20T17:05:08.133Z)

Now the error changed ? it now complains about `convention\_summary`, `code\_changes`, `phases` (and NOT `layout\_tree` since I moved it first). This confirms the parser is truncating my arguments after the FIRST parameter's content, because something in the value text is breaking the parse. The first param now (`layout\_tree`) survives; everything after the first param's value is being dropped. The breaking character is likely the slash sequences or commas, but more likely the issue is that each `` open of the next param is being mis-read.

The real fix: the harness is fine with multiple params normally. The breakage tracks with WHICHEVER param is parsed first surviving. This points to a single bad character inside the SECOND parameter every time. The common element across attempts in the 2nd-onward position... Let me try putting ALL content into the required fields only, each as simple single-line strings, minimal punctuation, to isolate.

42. user (2026-06-20T17:05:29.014Z)

Output does not match required schema: root: must have required property 'layout_tree', root: must have required property 'code_changes', root: must have required property 'phases'

43. assistant (2026-06-20T17:05:36.272Z)

Now it's back to flagging `layout\_tree, code\_changes, phases` ? meaning `convention\_summary` (first param this time) survived, and the rest got dropped. So whatever param comes SECOND is consistently breaking. This time the second param was `layout\_tree`. The breaking element must be a character in the value text. The common suspect across all my second-position values: parentheses `()` and the slash in `root/VIDEO\_ID`.

Let me test a hypothesis directly: submit ONLY the three problematic required fields with ultra-plain values (no parens, no slashes, no brackets except JSON), to see if it validates.

44. user (2026-06-20T17:05:50.750Z)

Output does not match required schema: root: must have required property 'layout_tree', root: must have required property 'code_changes', root: must have required property 'phases'

45. assistant (2026-06-20T17:05:59.677Z)

Even with ultra-plain values it fails on the same three fields, and `convention\_summary` (first param) survives. The pattern is now unmistakable: only the FIRST parameter of my tool call is being parsed; everything after is dropped. This is not about the value content ? it is about how the parameters after the first are being serialized in my output.

The harness wraps each parameter in `` tags. If my parameters are being collapsed, the most likely cause is that I have a trailing `</invoke>` artifact being injected. Let me try a completely different tactic: put EVERYTHING into a single first parameter is impossible (schema needs distinct keys).

Let me retry but order the parameters so the three array/string required ones come first, and critically, verify there is no stray closing tag. I'll emit exactly 9 parameters, nothing after the last.

46. user (2026-06-20T17:06:13.252Z)

Structured output provided successfully

47. assistant (2026-06-20T17:06:19.012Z)

The structured output was accepted. The earlier failures were caused by stray closing tags I had been appending after the final parameter, which corrupted the tool-call XML so only the first parameter survived parsing. Removing those and emitting clean parameter blocks fixed it.

The design has been returned via StructuredOutput: a single-folder-per-video workspace convention rooted at a storage URI (so local and cloud share one code path and identical relative layout), keyed by the MD5 video_id, built through a new `backend/utils/workspace.py` helper, with the DB holding metadata only plus a nullable `videos.workspace\_uri` pointer, introduced default-OFF behind a flag and rolled out in six smallest-safe-first phases.